IFT2255 — Génie logiciel (A26)

Projet — Phase 2

Conception de la plateforme de planification des études

Équipe 22

Nada Chiki (20261599)
Ismail Mohib (20274803)
Hamza Khalil (20259237)
Mohamed Driss (20205703)

Enseignant : Louis-Edouard Lafontant

Remis le 14 novembre 2026

Dépôt : https://github.com/Nada-ch12/IFT2255-equipe22

Table des matières

Table des matières 2
1. Introduction 4
2. Révisions des artefacts de la phase 1 5
Tableau des révisions 5
R1 — Fiches 1 et 3 corrigées 5
R2 — Matrice des droits des acteurs 6
R3 et R4 — Modèle d'états unique et flux A2.1 / A2.2 redessinés 7
R5 — Authentification et autorisation 10
R6 — Stratégie de synchronisation des données 11
R7 — Mécanisme des notifications 12
R8 — Validation des règles utilisées pour la démonstration 12
3. B1 — Modèle C4 niveau 3 14
1. Conteneur décomposé et justification 15
2. Composants et responsabilités 15
3. Matrice de traçabilité des cinq flux A2 16
4. Correspondance composants ↔ classes de B2 18
4. B2 — Diagramme de classes 20
1. Les 15 classes de la vue principale 22
2. Classes absentes du modèle du domaine 23
3. Écarts entre le modèle du domaine et la conception 23
4. Choix de conception à défendre 23
5. B3 — Spécifications de tests 25
1. Critères d'acceptation par exigence 25
2. Jeux de données 26
3. Cas de test 27
4. Couverture 33
6. B4 — Modèle de données 34
1. Les trois familles de données 34
2. Données d'échange 34
3. Données de préservation (conservées d'une visite à l'autre) 34

4. Données de présentation (calculées, jamais stockées)..... 35
5. Parcours de deux données importantes, de la source à l'affichage..... 36
7. Défense des choix..... 38
8. Déclaration de l'usage des outils d'IA..... 40
9. Répartition du travail..... 43
10. Annexe..... 44

(Dans Word : clic droit sur la table, puis « Mettre à jour les champs ».)

1. Introduction

La phase 1 a produit une analyse de la planification des études et un premier modèle de la solution : des exigences E1 à E14 rattachées aux objectifs O1 à O6, des cas d'utilisation (A1), cinq flux d'activités (A2) et une architecture C4 de niveaux 1 et 2 (A3).

La phase 2 passe de l'analyse à la conception. Ce rapport présente :

- **les révisions des artefacts de la phase 1** : les corrections demandées lors de l'évaluation et celles que la conception a révélées, chacune avec sa raison ;
- **B1**, le modèle C4 de niveau 3, avec la matrice de traçabilité des cinq flux A2 ;
- **B2**, le diagramme de classes et ses écarts avec le modèle du domaine ;
- **B3**, les spécifications de tests et leurs critères d'acceptation ;
- **B4**, le modèle de données ;
- **la défense de nos principaux choix**, puis la déclaration de l'usage des outils d'IA et la répartition du travail.

Un fil conducteur traverse tous les artefacts : **la plateforme dépend de sources qu'elle ne contrôle pas.** L'API Planifium ne comprend pas tous les textes d'exigences, ses places sont un instantané daté, et le calendrier ou le relevé peuvent être absents. Notre conception ne transforme jamais une information manquante en validation : elle la conserve avec sa date et sa provenance, et l'affiche comme incertaine.

Les sources des diagrammes (PlantUML), les fichiers de travail et le prototype de la phase 1 sont dans le dépôt : https://github.com/Nada-ch12/IFT2255-equipe22

2. Révisions des artefacts de la phase 1

L'énoncé de la phase 2 demande de signaler chaque révision des artefacts de la phase 1, avec la raison qui la motive. Les révisions viennent de deux sources : l'évaluation de la phase 1 (correctrice : Roxana Pena Mendieta) et ce que la conception de la phase 2 a révélé.

Tableau des révisions

#	Artefact	Révision	Raison
R1	A1 – Fiches 1 et 3	La Fiche 1 se termine maintenant sur une demande enregistrée « en attente » ; la confirmation relève de la Fiche 3.	Retour de la correctrice : la Fiche 1 annonçait « demandé puis confirmé », alors que A2.1 se termine sur « en attente ».
R2	A1 – Acteurs	Ajout d'une matrice des droits : qui peut lire, créer ou modifier chaque donnée.	Retour : les droits des acteurs qui modifient des données institutionnelles ou publient des offres n'étaient pas assez précis.
R3	A2.1, A2.2	A2.1 et A2.2 sont redessinés avec les noms d'états du modèle unique ; « annulée » (offre refusée par l'étudiant) devient « déclinée ».	Retour : A2.1 devait être aligné avec la Fiche 1. En conception, nous avons vu que « annulée » désignait deux situations différentes.
R4	Modèle d'états	Un seul diagramme d'états d'une demande, utilisé dans A1, A2, B1, B2, B3 et B4.	Retour : « cohérence du modèle d'états ».
R5	A3 – C4 niveaux 1 et 2	Ajout de l'authentification (fournisseur d'identité de l'Université) et de l'autorisation (contrôle d'accès dans l'API).	Retour : absentes de l'architecture cible.
R6	A3 – C4 niveau 2	Stratégie de synchronisation définie pour chaque source ; la source externe a toujours raison.	Retour : stratégie non définie. Planifium est en plus une source imparfaite (voir ci-dessous).
R7	A3 – C4 niveaux 1 et 2	Mécanisme des notifications : création, stockage sans doublon, affichage dans l'application et courriel.	Retour : mécanisme réel pas clair.
R8	Exigences, A2.3, A2.5	Les règles utilisées pour la démonstration sont vérifiées auprès des sources officielles ; chacune est marquée validée, corrigée ou hypothèse.	Retour : valider ces règles avant la phase suivante.

R1 — Fiches 1 et 3 corrigées

Fiche 1 — Demander une activité

- **But :** Soumettre une demande pour obtenir une place dans une activité (cours, projet, stage, séminaire) offerte pendant la session en cours.
- **Acteur principal :** personne étudiante.

- **Déclencheur :** l'étudiant clique sur « Demander » sur la carte d'une offre.
- **Préconditions :** l'étudiant est authentifié ; une case du plan correspond au type d'activité ; l'activité a au moins une place libre selon la dernière synchronisation.
- **Postcondition (succès) :** La demande est enregistrée au statut « en attente » et peut ensuite évoluer selon la réponse du tiers (Fiche 3). Les crédits et la charge s'affichent « en demande », sans être comptés comme confirmés.
- **Postcondition (échec) :** la demande n'est pas enregistrée et l'étudiant voit le motif (contrainte bloquante, case indisponible, places épuisées).
- **Dépendances :** inclut « Vérifier les contraintes du programme » (Fiche 2).

L'ancienne postcondition (« la demande passe à demandé puis à confirmé ») mélangeait deux cas d'utilisation. La confirmation dépend d'un tiers : elle n'est donc pas un résultat de la Fiche 1.

Fiche 3 — Recevoir une réponse à une demande (postconditions ajustées pour suivre la Fiche 1)

- **Postcondition (succès) :** la demande passe à « offre reçue », puis à « confirmée » quand l'étudiant accepte l'offre ; crédits et charge sont alors comptés comme confirmés.
- **Postcondition (échec) :** la demande passe à « refusée » (avec le motif), à « expirée » (pas de réponse avant l'échéance) ou à « déclinée » (l'étudiant refuse l'offre) ; l'étudiant est notifié.

R2 — Matrice des droits des acteurs

Principe : **la plateforme ne modifie jamais une donnée institutionnelle.** Les données du Registrariat, du répertoire des cours et du calendrier sont des copies en lecture seule, avec leur provenance et leur date d'import.

Donnée	Personne étudiante	Personne qui encadre	Laboratoires / organisations	Conseil de programme	Connecteurs d'import
Profil et relevé importé	Lire et modifier **le sien**	—	—	—	Écrire (import)
Plan, intentions	Créer, modifier **le sien**	—	—	—	—
Demande	Créer, annuler **la sienne**	Répondre aux demandes **qui lui sont adressées**	Répondre aux demandes **sur ses offres**	Lire (statistiques)	—
Offre (projet, stage, poste)	Lire	Créer, modifier, fermer **ses** offres	Créer, modifier, fermer **ses** offres	Lire ; retirer une offre non conforme	—
Règles du programme	Lire	Lire	Lire	**Seul à créer et modifier** ; chaque modification est datée et versionnée	—
Cours, exigences, dates officielles	Lire	Lire	Lire	Lire	**Seuls à écrire** (copie de la source)
Statut officiel d'inscription	Lire	—	—	—	Lire ; jamais écrit par la plateforme

- Les comptes des organisations externes sont créés sur invitation, validée par le conseil de programme.

- Chaque écriture est journalisée (qui, quand).
- Une donnée institutionnelle qui semble fausse n'est pas corrigée dans la plateforme : elle est signalée, et l'étudiant est orienté vers la source.

R3 et R4 — Modèle d'états unique et flux A2.1 / A2.2 redessinés

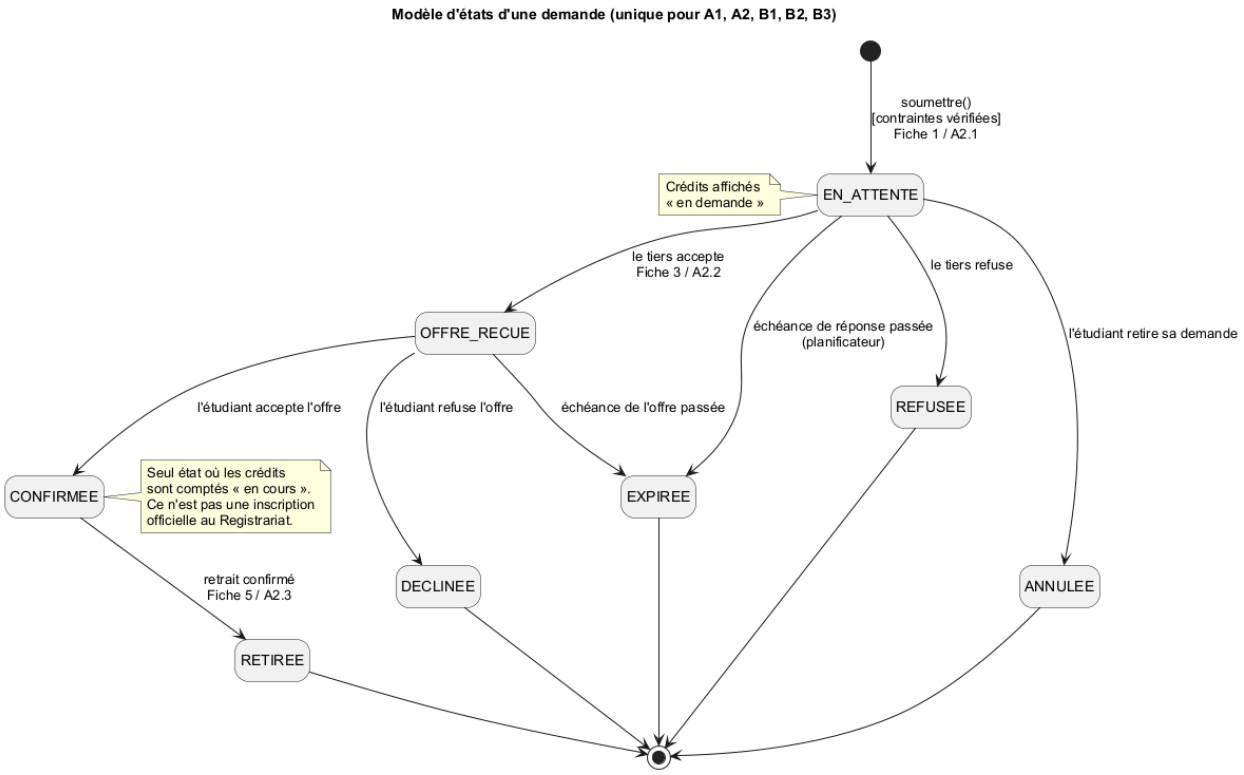


Figure 1 — Modèle d'états d'une demande

Artefact	Ce qu'il couvre	État de fin
Fiche 1 / A2.1	Soumettre la demande après vérification des contraintes	EN_ATTENTE
Fiche 3 / A2.2	Réponse du tiers et décision de l'étudiant	CONFIRMEE, REFUSEE, EXPIREE ou DECLINEE
Fiche 5 / A2.3	Retrait d'une activité confirmée	RETIREE

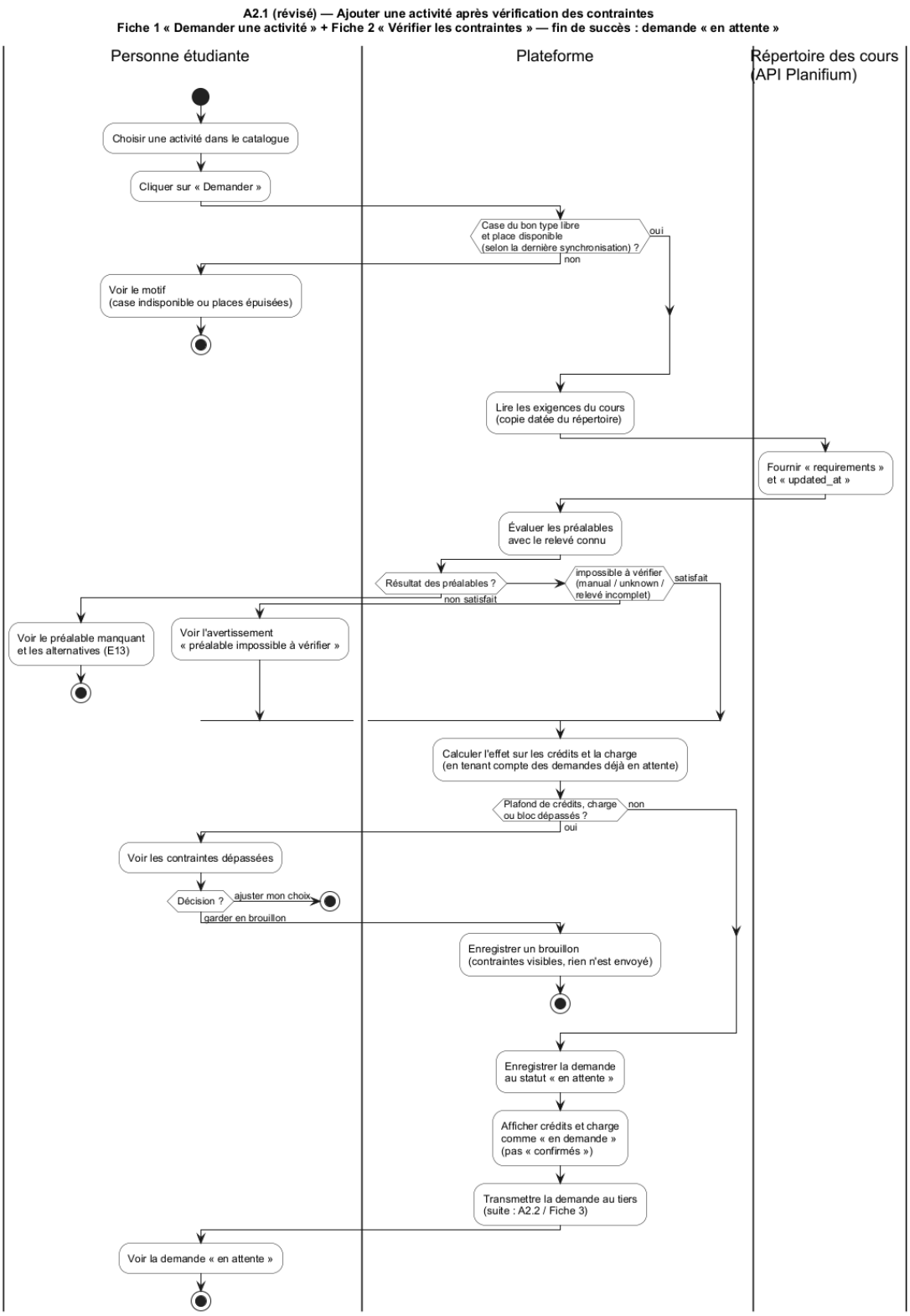


Figure 2 — A2.1 révisé : ajouter une activité après vérification des contraintes

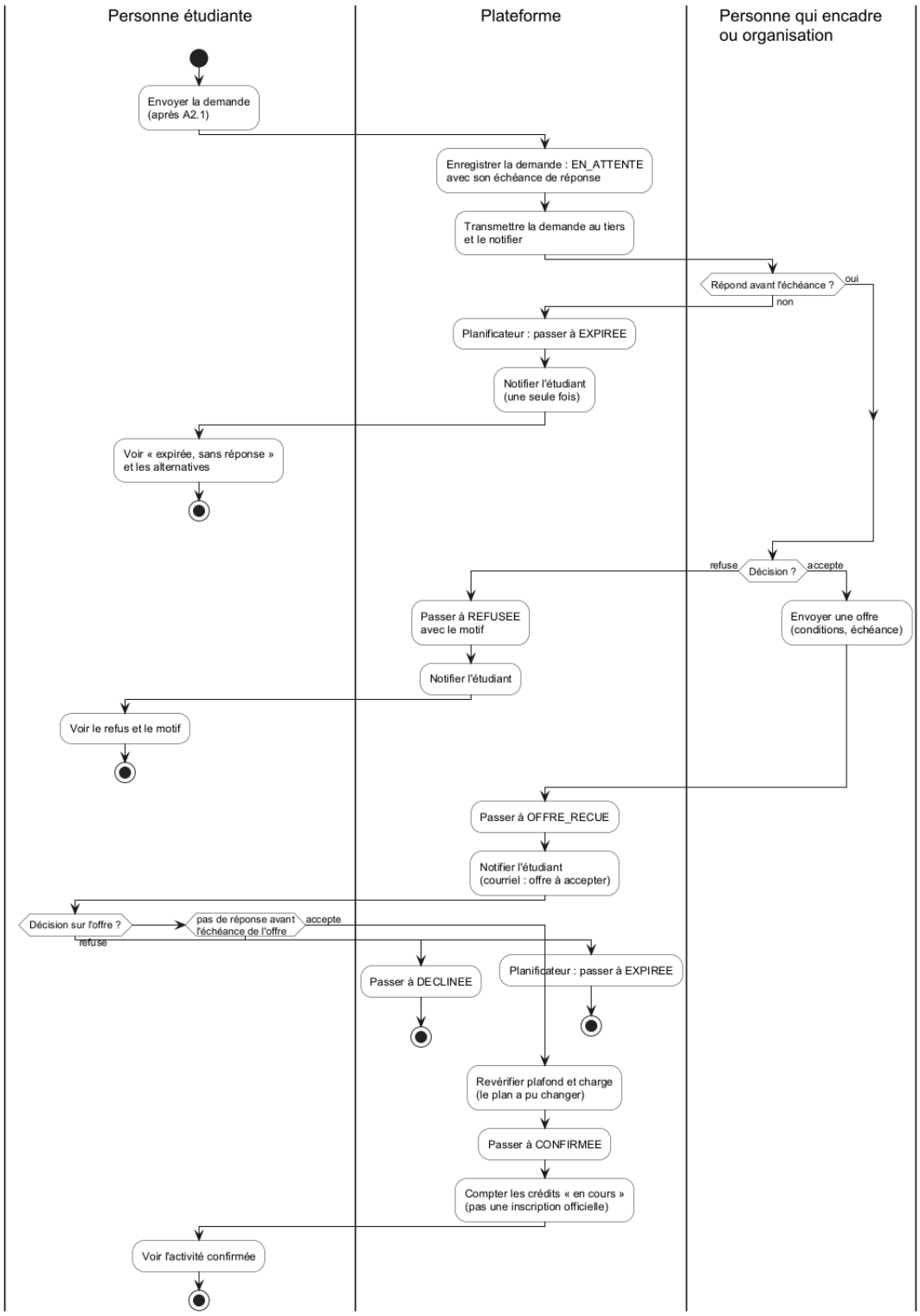


Figure 3 — A2.2 révisé : demander une place et répondre à une offre

Dans A2.1, un préalable « impossible à vérifier » (exigence non analysée par Planifium ou relevé absent) produit un avertissement, et jamais une validation. C'est la correction de l'observation IFT1025 de la phase 1 (S5).

R5 — Authentification et autorisation

- **Niveau 1 :** nouveau système externe, le **fournisseur d'identité de l'Université** (connexion unique).
- **Niveau 2 :** l'application web redirige l'utilisateur vers le fournisseur d'identité et reçoit un jeton. À chaque requête, l'API vérifie le jeton (**authentification**), puis le rôle et le propriétaire de la donnée selon la matrice R2 (**autorisation**). Exemple : un encadrant ne peut répondre qu'aux demandes qui lui sont adressées.
- Les rôles et le journal des actions sont conservés dans la base de données.
- En phase 2, ce mécanisme devient le composant **Contrôle d'accès** (B1) et les classes `ControleAcces` et `Utilisateur` (B2).

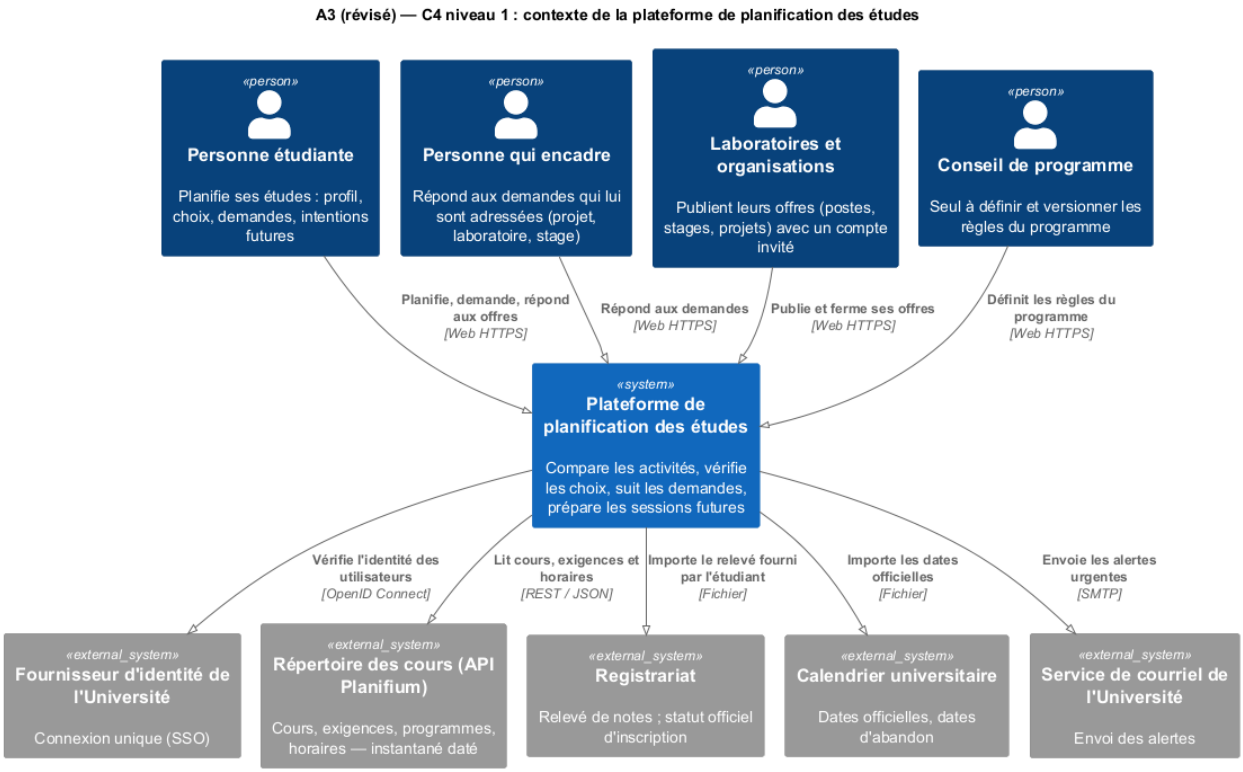


Figure 4 — A3 révisé : C4 niveau 1, contexte

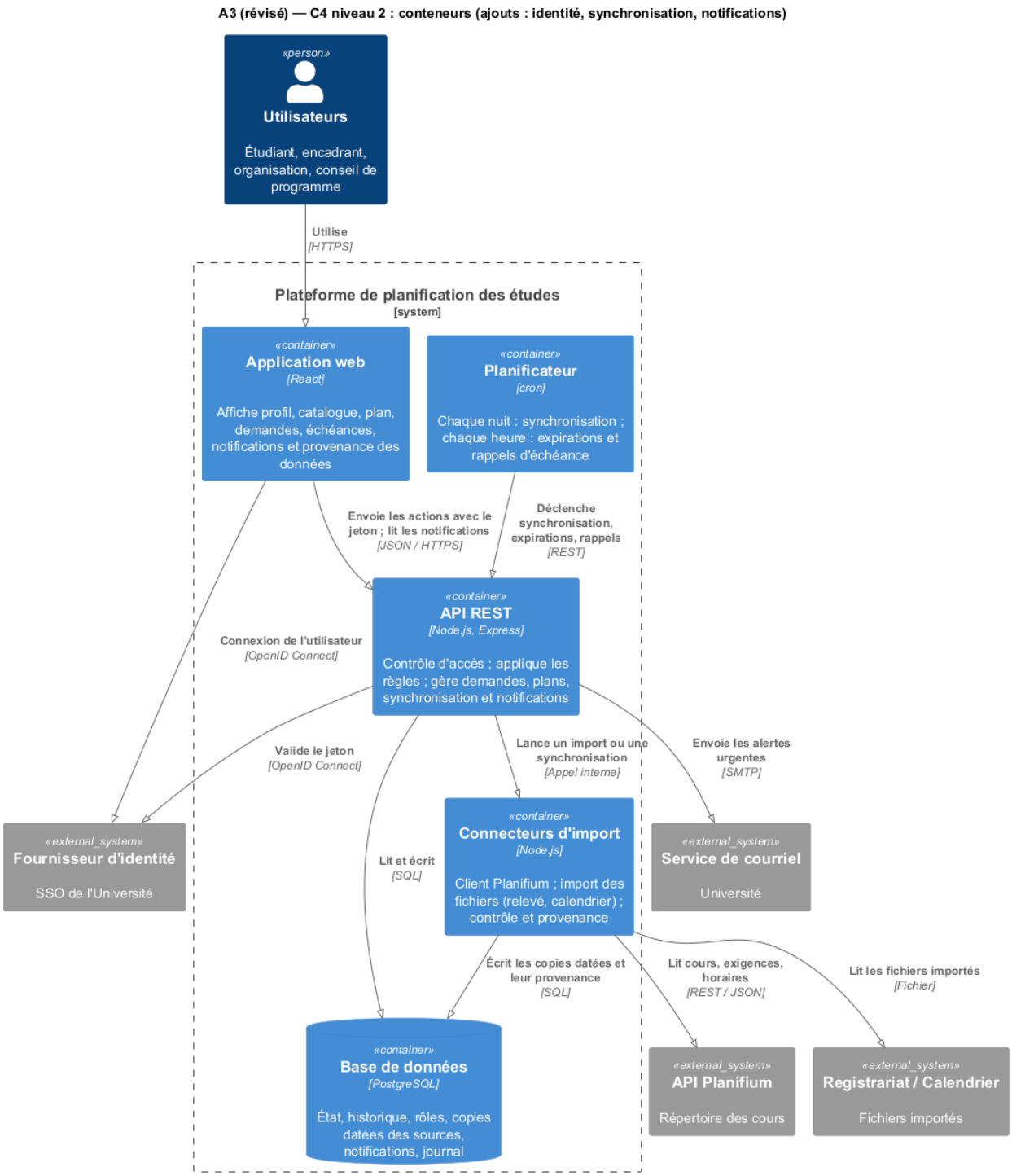


Figure 5 — A3 révisé : C4 niveau 2, conteneurs

R6 — Stratégie de synchronisation des données

Principe : **la source externe a toujours raison.** La plateforme garde une copie locale datée et l'affiche avec sa provenance. Si la source ne répond pas, la plateforme utilise la dernière copie et le dit (« données du 3 nov. »).

Source	Données	Comment	Quand	Si la source est indisponible ou incomplète
API Planifium (répertoire des cours)	Cours, exigences (`requirements`), horaires, places	Lecture par le Client Planifium (*pull*)	Chaque nuit (planificateur) et sur demande	Dernière copie gardée et marquée « à vérifier » ; une exigence non analysée donne « inconnu », jamais « satisfait »
Registrariat	Relevé de notes	Fichier importé par l'étudiant	Quand l'étudiant le décide	Le relevé reste daté de son import
Calendrier universitaire	Dates officielles, dates d'abandon	Import de fichier	Début de session, puis chaque semaine	Une date absente s'affiche « date non publiée »
Offres des organisations	Places, périodes	Saisie dans la plateforme	En temps réel	La plateforme est la source de vérité

Quand une synchronisation détecte un changement (cours retiré, offre complète), le Service Synchronisation marque les intentions touchées « à réexaminer » et crée une notification (A2.4, Fiche 10).

Ce que le guide de l'API (section « Limites à connaître ») dit de Planifium, et notre réponse :

- environ 90 % des textes d'exigences sont compris (`status: "exact"`) ; les autres sont `manual` ou non interprétés, et certaines conditions donnent `unknown` → résultat INCONNU, jamais SATISFAIT (E3) ;
- `prerequisite_courses` ne distingue pas le « et » du « ou » → nous utilisons `requirements` ;
- les places (`enrolled`, `capacity`) sont un instantané daté par `updated_at` → la date est toujours affichée ;
- Planifium ne gère pas les inscriptions → une confirmation dans la plateforme n'est pas une inscription officielle ;
- aucun quota ni garantie de disponibilité n'est décrit → délai maximal d'appel et repli sur la dernière copie.

R7 — Mécanisme des notifications

1. **Création :** le Service Demandes crée une notification à chaque changement d'état (offre reçue, refus, expiration). Le Service Synchronisation en crée une à chaque changement d'offre, et le Service Échéances et retraits crée les rappels 7 jours avant une échéance, à l'appel du planificateur.
2. **Stockage :** chaque notification est enregistrée dans la base (destinataire, événement, clé unique, lue / non lue). La clé unique empêche les doublons quand le planificateur repasse.
3. **Livraison :** dans l'application (notifications non lues affichées à l'ouverture, puis rechargées régulièrement) et par courriel, via le service de courriel de l'Université, pour les événements urgents (offre à accepter, échéance proche, expiration).
4. **Ce qui n'est pas promis :** pas de notification instantanée (*push*). Dans le prototype, seul l'affichage dans l'application existe ; le courriel est conçu, pas réalisé.

R8 — Validation des règles utilisées pour la démonstration

Règle	Où	Source consultée	Statut

Temps plein = minimum 12 crédits par trimestre	A2.3, E9	Règlement du baccalauréat en informatique, art. 6.3 (admission.umontreal.ca)	**Validée**
Plafond de 18 crédits par trimestre	Fiche 2, A2.1	Même article 6.3 (« maximum de 18 crédits par trimestre »)	**Validée** pour le bacc en informatique ; le plafond est un paramètre de `Programme`, puisqu'il varie selon le programme
Plafond de 15 crédits du profil 8 h	Prototype	Aucune source universitaire	**Corrigée** : retirée. Le plafond vient du programme ; l'écart entre la charge et la disponibilité devient un avertissement séparé (`RegleChargeHebdo`)
Abandon avec frais après le 16 septembre	A2.3	Calendrier universitaire A26 et Centre étudiant	**Corrigée / paramétrée** : les dates exactes varient par cours au Centre étudiant (cours intensifs ou trimestriels) ; elles sont donc traitées comme des paramètres importés (`Echeance`) et non comme des constantes fixes
Retrait refusé après le 6 novembre, orientation vers le Registrariat	A2.3	Calendrier universitaire A26 et Centre étudiant	**Corrigée / paramétrée** : les dates exactes varient par cours au Centre étudiant (cours intensifs ou trimestriels) ; elles sont donc traitées comme des paramètres importés (`Echeance`) et non comme des constantes fixes
Alerte 7 jours avant une échéance	A2.5	—	**Choix de conception** de l'équipe, présenté comme tel
Délai d'expiration d'une demande	A2.2	Aucune règle observée (Coopérative Volta, sem. 10)	**Paramètre** : chaque demande porte son `echeanceReponse` ; aucune durée uniforme supposée
Préalable de IFT1025	Fiche 2, A2.1, B3	Réponse réelle de l'API Planifium (vérifiée le 2026-11-10)	**Corrigée** : « IFT1015 ou IFT1016 » (`any`), et non IFT1015 seul. Les préalables de IFT3395, IFT2255 et STT1700 ont aussi été vérifiés (B3)

3. B1 — Modèle C4 niveau 3

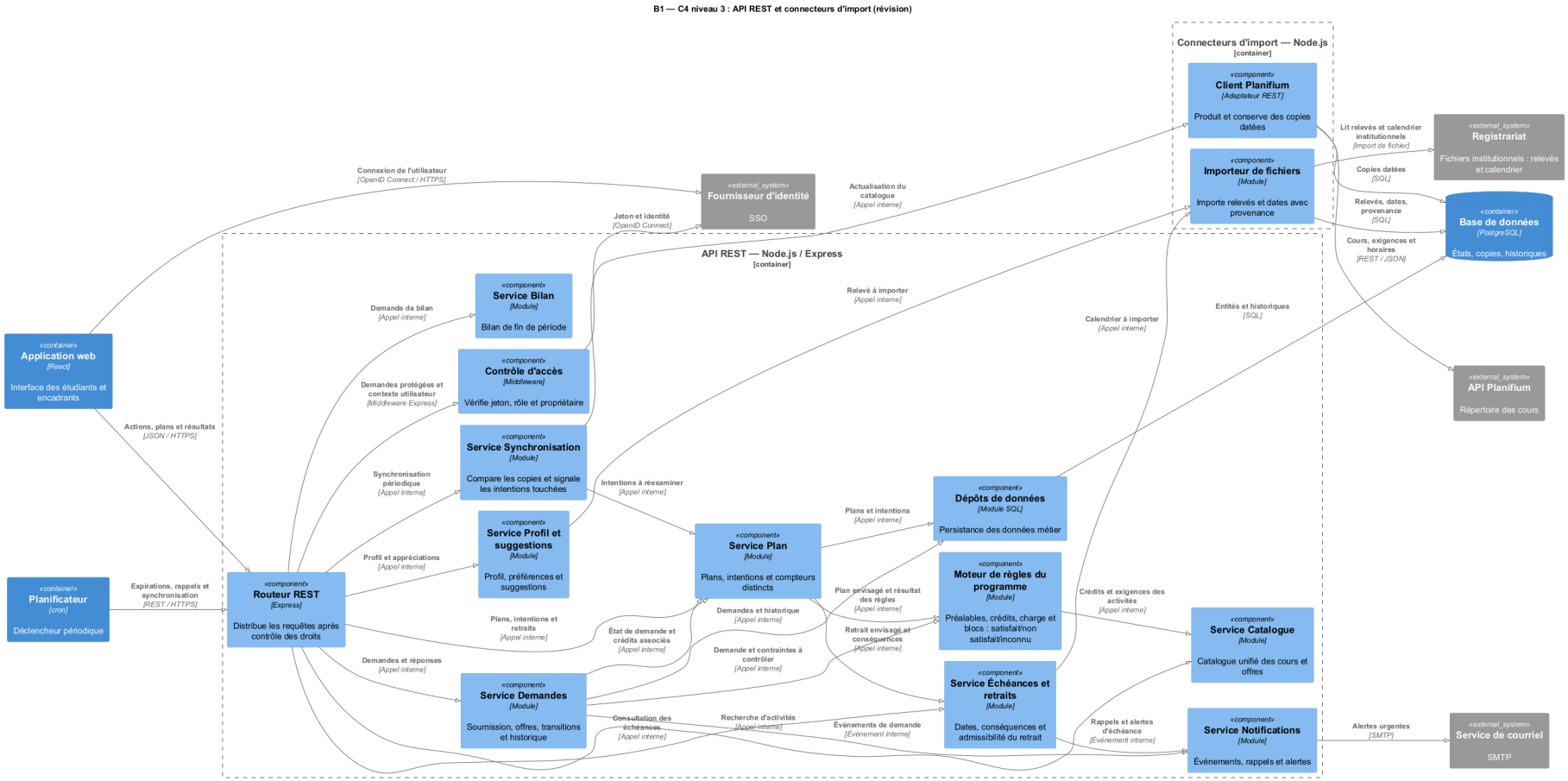


Figure 6 — C4 niveau 3 de l'API REST et des connecteurs d'import

1. Conteneur décomposé et justification

Nous décomposons l'**API REST**, et plus brièvement les **Connecteurs d'import**.

Pourquoi c'est l'API qui porte la logique de planification :

5. **Les règles ne doivent pas pouvoir être contournées.** L'application web tourne dans le navigateur de l'utilisateur : on ne peut pas lui faire confiance pour appliquer le plafond de crédits ou les préalables. Dans le prototype de la phase 1, ces règles étaient dans `rules.js`, côté navigateur, et les essais ont montré des dépassements (25/18 crédits) qui n'étaient pas bloqués.
6. **La planification a besoin de toutes les données en même temps** : le relevé, le plan, les demandes en attente, les copies de Planifium et les règles versionnées. Seule l'API a accès à la base et aux connecteurs.
7. **Plusieurs déclencheurs utilisent la même logique** : l'étudiant (application web), l'encadrant (réponse à une demande) et le planificateur (expiration, synchronisation). Placer les règles dans l'API garantit un seul comportement pour tous (pas de règle dupliquée).
8. **Les connecteurs restent à part**, parce que Planifium est une source que nous ne contrôlons pas : lente, parfois indisponible, avec des données incomplètes. Si Planifium tombe, l'API continue de fonctionner avec la dernière copie datée.

2. Composants et responsabilités

Composant	Responsabilité (une phrase)	Exigences
Routeur REST	Reçoit chaque requête HTTP et la dirige vers le bon service.	—
Contrôle d'accès	Vérifie le jeton et applique la matrice des droits par rôle et par propriétaire.	R5 (révision)
Service Profil et suggestions	Gère le profil, les préférences, les appréciations et explique les suggestions.	E2, E12
Service Catalogue	Réunit les cours copiés de Planifium et les offres des organisations en un catalogue commun.	E1, E13
Service Plan	Gère le plan de session, les intentions futures et les compteurs de crédits séparés.	E4, E10, E11
Moteur de règles du programme	Évalue préalables, plafond, charge, blocs et temps plein avec un résultat à trois valeurs.	E3, E4, E5, E9
Service Demandes	Fait évoluer chaque demande selon le modèle d'états et garde son historique.	E6
Service Échéances et retraits	Fournit les dates utiles et calcule les conséquences d'un retrait.	E7, E8
Service Notifications	Crée une notification par événement, sans doublon, et envoie les urgentes par courriel.	E6, E7, E11
Service Synchronisation	Compare les nouvelles copies aux anciennes et lance le réexamen des plans touchés.	E11
Service Bilan	Produit le bilan de fin de période : satisfait, incomplet, incertain.	E14
Dépôts de données	Lit et écrit les entités et leur historique dans la base.	—

Client Planifium *(connecteurs)*	Appelle Planifium et traduit ses réponses en copies datées avec leur statut.	E1, E3
Importeur de fichiers *(connecteurs)*	Lit le relevé et le calendrier, vérifie leur format et note leur provenance.	E3, E7

Toutes les relations du diagramme ont une étiquette (verbe + ce qui circule).

3. Matrice de traçabilité des cinq flux A2

Convention de lecture

- **App** = conteneur Application web (React) ; **Routeur** = composant Routeur REST de l'API ; **Accès** = composant Contrôle d'accès (middleware Express).
- Les flèches → indiquent la succession logique des responsabilités. Elles ne signifient pas toutes un appel direct : une vérification peut retourner son résultat au composant appelant.
- Les opérations protégées passent par le middleware **Contrôle d'accès**, même lorsque son intervention est indiquée une seule fois en début de parcours.
- Les décisions et variantes proviennent des flux A2 du rapport de phase 1 ; cette matrice décrit la **conception visée**, et non le comportement actuellement implémenté dans le prototype.
- Les opérations de persistance métier sont réalisées par **Dépôts de données → Base de données (PostgreSQL)**. Les copies de Planifium sont produites par **Client Planifium**.

Matrice principale

Flux A2	Déclencheur et étapes métier	Parcours principal dans B1	Cas alternatifs et responsables	Données / résultat
A2.1 — Ajouter une activité après vérification des contraintes	L'étudiant choisit une activité ; le système vérifie les préalables, le plafond de crédits et la charge, puis soumet la demande autorisée.	**App → Routeur → Accès → Service Demandes → Moteur de règles du programme ↔ Service Catalogue → Service Demandes → Service Plan → Dépôts de données → PostgreSQL.** Le Service Demandes peut publier un événement vers **Service Notifications**.	**NON_SATISFAIT** : blocage et explication, sans demande enregistrée. **INCONNU** (condition absente/non analysable) : ne pas traiter comme satisfait ; afficher l'incertitude ou réclamer une vérification. L'étudiant peut modifier/annuler son choix.	Activité, préalables, relevé, crédits demandés/confirmés, résultat de règle, état de la demande.
A2.2 — Demander une place et répondre à une offre	Demande transmise ; encadrant répond ; offre reçue ; étudiant accepte ; statut confirmé.	**Étudiant/encadrant : App → Routeur → Accès → Service Demandes → Service Plan**, puis **Service Demandes → Dépôts de données → PostgreSQL** et **Service Demandes → Service**	**Refus** : Service Demandes passe à refusée et notifie. **Expiration** : Planificateur → Routeur → Accès → Service Demandes (expirée) → Service Notifications. **Offre déclinée** : état annulé ; pas de confirmation. Transition refusée si illégale par `Demande.transitionner()`.	Statut et historique d'une demande, interlocuteur, échéance de réponse, notification, crédits confirmés distincts des demandes en attente.

		Notifications. Pour l'acceptation, **Service Demandes → Moteur de règles** si les contraintes doivent être recontrôlées.		
A2.3 — Retirer une activité en comprenant les conséquences	L'étudiant demande un aperçu ; le système consulte dates/frais, calcule crédits et temps plein ; l'étudiant confirme ou annule.	**App → Routeur → Accès → Service Plan → Service Échéances et retraits** ; en parallèle logique **Service Plan → Moteur de règles du programme**. L'aperçu revient à l'**App**. Après confirmation autorisée : **App → Routeur → Accès → Service Plan → Dépôts de données → PostgreSQL** ; si la demande doit changer d'état, **Routeur → Service Demandes → Dépôts de données**.	**Annulation** : aucune écriture. **Après le 16 septembre** : montrer frais et conséquences connus avant confirmation. **Après le 6 novembre**, selon la règle visée de la phase 1 : refuser le retrait et orienter vers le Registrariat. **Date/frais inconnus** : le signaler, sans prétendre « sans frais ».	Date limite, conséquences financières connues, crédits/charge/temps plein après retrait, confirmation explicite, statut et historique.
A2.4 — Réexaminer une intention pour une session future	L'étudiant enregistre une intention distincte d'une inscription ; une synchronisation ultérieure découvre un changement d'offre.	**Création : App → Routeur → Accès → Service Plan → Moteur de règles / Service Catalogue (selon besoin) → Service Plan → Dépôts de données → PostgreSQL. Réexamen : Planificateur → Routeur → Accès → Service Synchronisation → Client Planifium → API Planifium** ; **Client Planifium → PostgreSQL** (copies datées), puis **Service Synchronisation → Service Plan** (intention à réexaminer).	Offre retirée, pleine ou données incertaines : marquer **à réexaminer** et prévenir l'étudiant via **Service Notifications** (relation `sync → notif` à ajouter si cette variante est modélisée comme appel direct). Une intention ne devient pas automatiquement confirmée.	Session visée, disponibilité, date/provenance de copie, statut de l'intention, notification de changement.
A2.5 — Consulter ses échéances	L'étudiant consulte ses dates ; le système distingue dates connues, dépassées et non publiées ; des	**Consultation : App → Routeur → Accès → Service Échéances et retraits** (dates conservées provenant de **Importeur de**	**Date non publiée** : afficher « inconnue/non publiée », sans inventer de date. **Date dépassée** : signaler l'échéance et les possibilités pertinentes. **Rappel J−7** uniquement si la date est connue et le rappel applicable ;	Date, source/provenance, état connu/inconnu/dépassé, rappel et notification.

	rappels sont déclenchés.	**fichiers → PostgreSQL**) → **App**. **Rappels : Planificateur → Routeur → Accès → Service Échéances et retraits → Service Notifications → Service de courriel** (si alerte urgente).	éviter les doublons.	

Vérification de cohérence avec le diagramme C4 niveau 3

La version actuelle du diagramme B1 comporte les relations nécessaires aux cinq flux, notamment : `dem → regles`, `regles → cata`, `dem → plan`, `plan → ech`, `plan → regles`, `dem → notif`, `sync → plan`, `sync → cliP`, `sync → notif`, `ech → depots`, `ech → notif`, `notif → depots`, `depots → db`, ainsi que les liens avec les systèmes externes (Planifium, Registrariat, fournisseur d'identité et service de courriel).

Précisions de lecture :

- Les parcours ci-dessus sont **logiques** : un résultat peut revenir au service appelant sans qu'une deuxième flèche inverse soit dessinée.
- Le contrôle d'accès est appliqué via un middleware Express aux opérations protégées avant leur exécution, y compris pour les opérations déclenchées par le planificateur.
- Les changements d'état de `Demande` sont persistés par les dépôts ; `Service Plan` maintient des compteurs distincts pour les demandes et les confirmations.
- La consultation des informations importées peut impliquer la base de données, sans que chaque lecture soit dessinée individuellement dans cette vue C4 de niveau 3.
- Les traitements de refus, d'annulation ou d'information inconnue sont documentés comme des branches métier, pas comme des composants supplémentaires.

Correspondance avec le rapport de phase 1

- A2.1 : exigences/préalables, plafond et charge ; contrainte inconnue non validée.
- A2.2 : attente, offre, acceptation, refus ou expiration, avec tiers et historique.
- A2.3 : conséquences avant décision, annulation possible, limites selon dates et statut de temps plein.
- A2.4 : intention future distincte, réexamen si l'offre change ou disparaît.
- A2.5 : dates officielles ou inconnues, échéance dépassée et alerte sept jours avant lorsque possible.

Document de revue à valider avec le diagramme et les choix de l'équipe ; ne pas le présenter comme une implémentation testée.

4. Correspondance composants ↔ classes de B2

Composant B1	Classes B2 (vue principale)	Classes B2 (annexe)
Contrôle d'accès	—	ControleAcces, Utilisateur, Role
Service Profil et suggestions	Etudiant	ServiceSuggestions, Appreciation
Service Catalogue	Activite, Cours, OffreActivite	—
Service Plan	PlanSession, Intention	Case, CompteurCredits
Moteur de règles du programme	Programme, RegleProgramme,	RegleChargeHebdo, RegleTempsPlein,

	ReglePlafondCredits, ReglePrealables, RegleBloc, MoteurRegles	ResultatRegle
Service Demandes	Demande, ServiceDemandes	ChangementEtat
Service Échéances et retraits	—	Echeance, ServiceEcheancesRetraits
Service Notifications	—	Notification, ServiceNotifications
Service Synchronisation	—	ServiceSynchronisation, RapportSynchro *(utilise CopiePlanifium sans la posséder)*
Service Bilan	—	ServiceBilan, Bilan
Client Planifium (connecteurs)	ClientPlanifium	CopiePlanifium *(produite et enregistrée par le Client Planifium)*
Dépôts de données	—	(une interface de dépôt par entité, non dessinée)

*À faire en équipe : vérifier que **chaque nom** ci-dessus est identique dans B1, B2, B3 et B4.*

4. B2 — Diagramme de classes

Diagrammes :

- Le modèle du domaine (concepts du problème) qui sert de point de comparaison ;

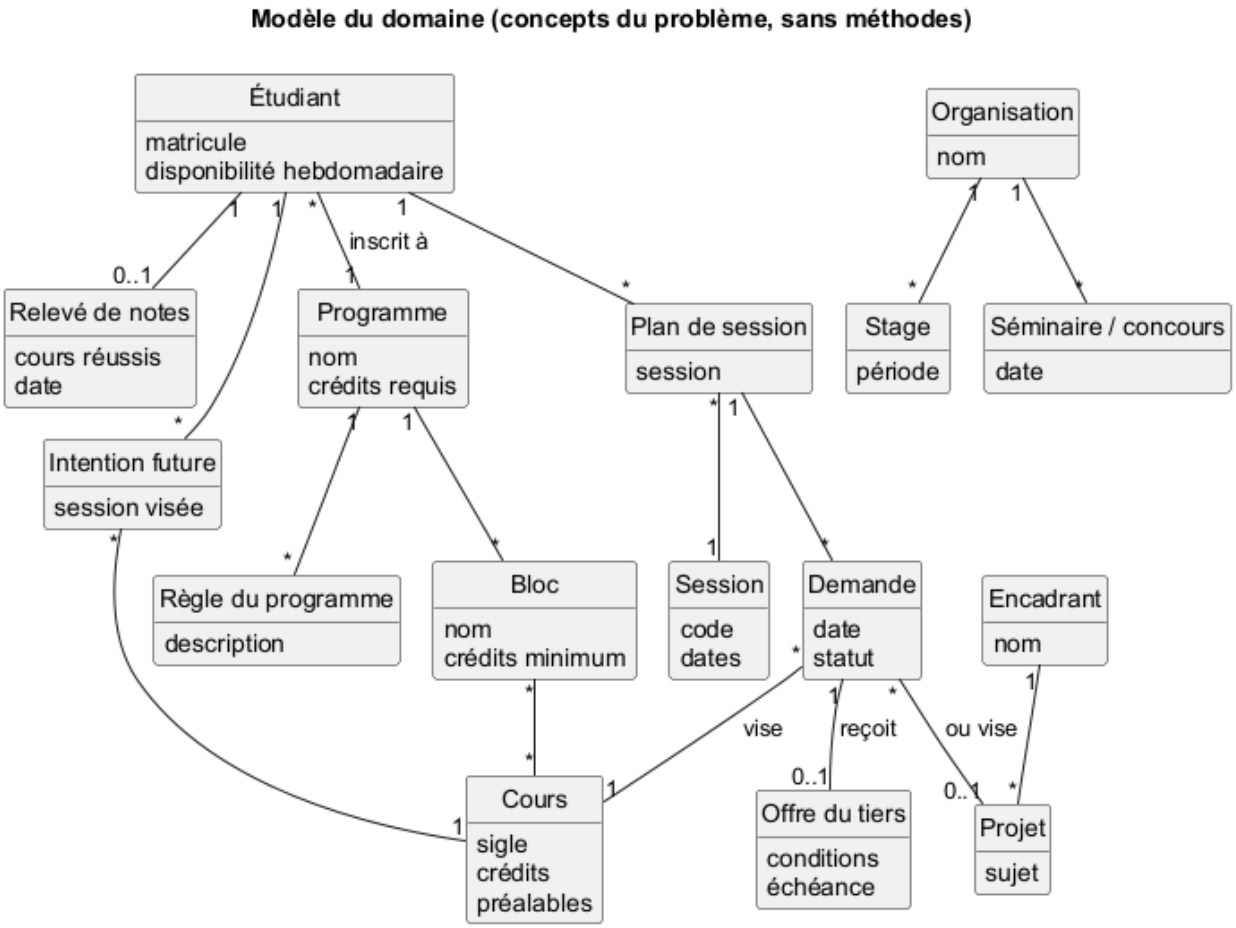


Figure 7 — B2 : modèle du domaine

- La **vue principale (15 classes)** ;

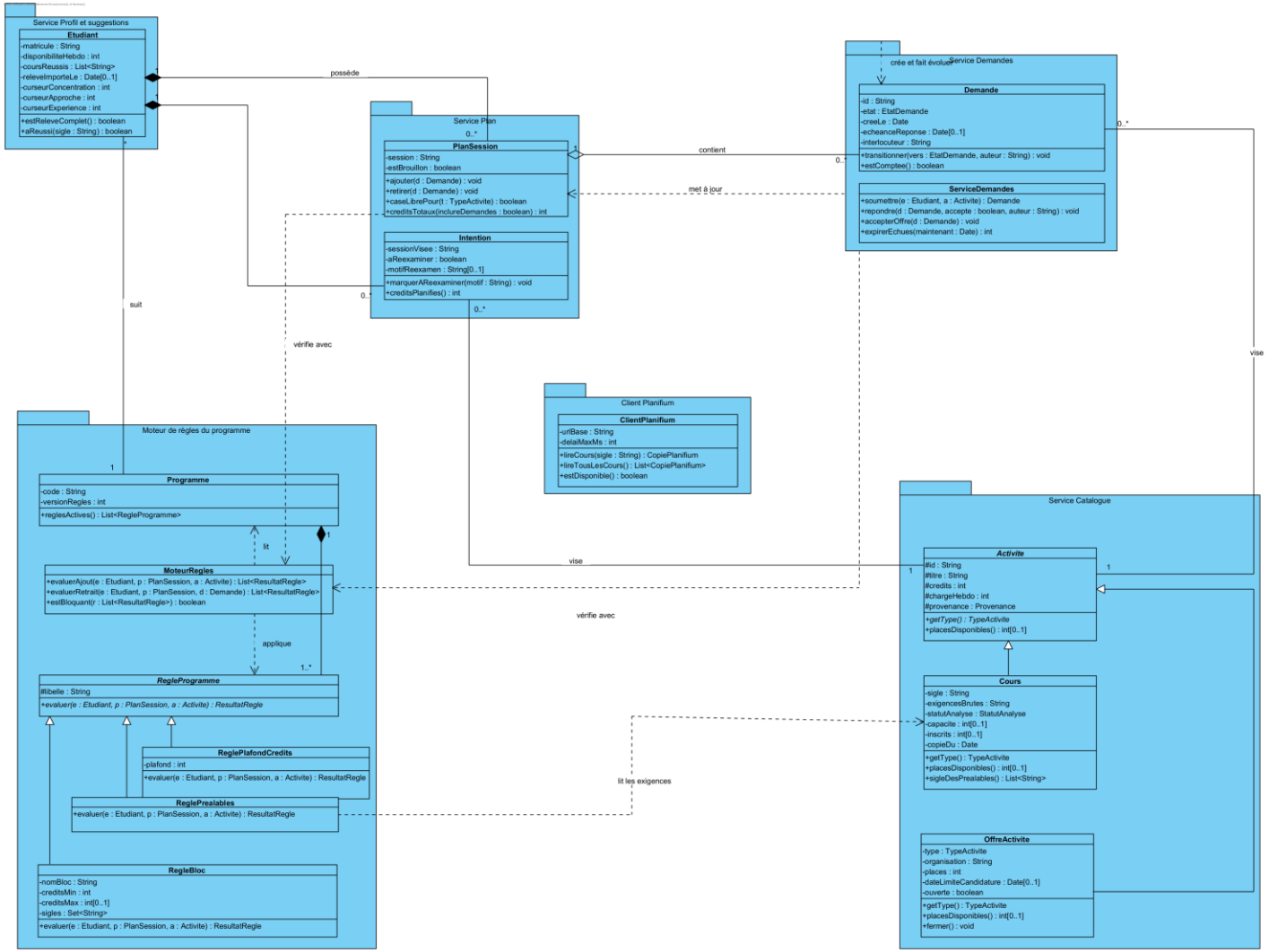


Figure 8 — B2 : diagramme de classes, vue principale (15 classes)

- L'annexe (énumérations et classes complémentaires), rangée dans les mêmes paquetages que la vue principale, c'est-à-dire les composants de B1 (voir l'annexe).

Décisions de nommage validées en équipe (6 nov.) :

- le composant s'appelle « Service Échéances et retraits » partout ; sa classe de service est `ServiceEcheancesRetraits` ;
- `CopiePlanifium` appartient aux **Connecteurs d'import** : le `ClientPlanifium` la produit et l'enregistre. Le `ServiceSynchronisation` la compare et le Service Catalogue lit sa date ; aucun des deux ne la possède.

Les paquetages du diagramme portent **le nom des composants de B1** : chaque classe se retrouve donc dans un composant.

1. Les 15 classes de la vue principale

Classe	Rôle	Composant B1
Etudiant	Profil, disponibilité et relevé connu de la personne.	Service Profil et suggestions
Activite *(abstraite)*	Ce qui est commun à toute activité : crédits, charge, provenance.	Service Catalogue
Cours	Cours copié de Planifium, avec ses exigences brutes et leur statut d'analyse.	Service Catalogue
OffreActivite	Projet, stage, poste ou séminaire publié par une organisation.	Service Catalogue
PlanSession	Plan d'une session : ses demandes et son état de brouillon ; ses crédits sont dans `CompteurCredits` (annexe).	Service Plan
Intention	Activité (`Activite`) visée pour une session future, qui peut être « à réexaminer ».	Service Plan
Programme	Programme suivi et version de ses règles.	Moteur de règles
RegleProgramme *(abstraite)*	Contrat commun : `evaluer()` renvoie un résultat à trois valeurs.	Moteur de règles
ReglePlafondCredits	Règle du plafond de crédits par session.	Moteur de règles
ReglePrealables	Règle des préalables, à partir des exigences copiées de Planifium.	Moteur de règles
RegleBloc	Règle d'un bloc : crédits minimum et maximum sur une liste de sigles.	Moteur de règles
MoteurRegles	Applique toutes les règles actives et dit si le résultat est bloquant.	Moteur de règles
Demande	Une demande et son état ; toute transition passe par `transitionner()`.	Service Demandes
ServiceDemandes	Soumet, fait répondre, fait accepter et fait expirer les demandes.	Service Demandes
ClientPlanifium	Adaptateur qui appelle Planifium et produit des copies datées.	Client Planifium (connecteurs)

Où vivent les règles du programme (exigence de l'énoncé) : plafond → `ReglePlafondCredits` ; préalables → `ReglePrealables` ; blocs → `RegleBloc`. En annexe : `RegleChargeHebdo` et

`RegleTempsPlein`. Ajouter une règle ne demande qu'une nouvelle sous-classe de `RegleProgramme` ; ni `MoteurRegles` ni `ServiceDemandes` ne changent.

2. Classes absentes du modèle du domaine

Classe	Pourquoi elle n'existe pas dans le domaine	Pourquoi la conception en a besoin
MoteurRegles	Dans le domaine, on parle de « règles », pas d'un objet qui les applique.	Un seul point d'entrée pour évaluer un ajout ou un retrait ; utilisé par le Service Demandes, le Service Plan et le planificateur (même comportement partout).
ServiceDemandes	Le domaine décrit la demande, pas l'orchestration de son cycle de vie.	Coordonne la vérification des règles, le changement d'état, l'historique et la notification.
ClientPlanifium	Planifium est un choix technique : le domaine parle seulement d'un « répertoire des cours ».	Isole une source imparfaite (lente, incomplète, datée). Si l'API change, une seule classe change.
(annexe) ResultatRegle, CopiePlanifium, Notification, ControleAcces	Ce sont des objets techniques de conception.	Résultat à trois valeurs, copie datée, notification sans doublon, autorisation.

3. Écarts entre le modèle du domaine et la conception

Écart 1 — Gestion des données externes

Dans la phase 1, nous avions prévu de récupérer les cours et leurs préalables depuis un répertoire externe, avec des connecteurs d'import. Dans notre conception, nous avons précisé cette gestion avec `ClientPlanifium`, qui possède la méthode `lireCours()`, et `CopiePlanifium`, qui conserve les données récupérées. Les attributs `aVerifier` et `statutAnalyse` permettent de signaler les informations qui nécessitent encore une vérification.

Écart 2 — Évaluation des règles du programme

Dans la phase 1, nous avions prévu de vérifier les contraintes du programme, comme les préalables, le plafond de crédits et la charge de travail, sans préciser leur mécanisme d'évaluation interne. Dans notre conception, nous avons introduit `MoteurRegles`, qui utilise plusieurs règles spécialisées héritant de `RegleProgramme`. Sa méthode `evaluerAjout()` retourne une liste de `ResultatRegle`, ce qui permet de connaître séparément le résultat de chaque vérification.

4. Choix de conception à défendre

Choix 1 — Isoler la communication avec Planifium

Nous avons choisi de passer par `ClientPlanifium` et de conserver une copie des données dans `CopiePlanifium`, plutôt que d'appeler l'API à chaque consultation. Cela permet d'utiliser les informations déjà récupérées lorsque Planifium est lent ou indisponible. Le compromis est que ces données peuvent devenir périmées, d'où les attributs `aVerifier` et `statutAnalyse`.

Choix 2 — Utiliser des règles spécialisées

Nous avons choisi de séparer les règles dans plusieurs classes plutôt que de mettre toutes les vérifications dans une seule grosse méthode. Les classes `ReglePrealables`, `ReglePlafondCredits` et `RegleBloc` héritent de `RegleProgramme` et utilisent la méthode `evaluer()`, ce qui permet au moteur de traiter différentes contraintes. Cette organisation facilite les modifications et l'ajout de nouvelles règles, mais elle demande de gérer davantage de classes.

Choix 3 — Séparer les demandes du plan de session

Nous avons choisi de garder `Demande` séparée de `PlanSession`, plutôt que de mettre directement les états des demandes dans le plan. La méthode `transitionner()` permet de faire évoluer une demande, tandis que `ChangementEtat` conserve l'historique de ses différents états. Cela facilite le suivi des réponses des tiers, mais demande de maintenir la cohérence entre les demandes et le plan lorsqu'une demande change d'état.

5. B3 — Spécifications de tests

Cette section contient 24 cas de test, avec des cas nominaux (N), limites (L) et d'échec (É).

> ***Source des données.*** *IFT1025, IFT3395, IFT2255 et STT1700 proviennent des réponses réelles de l'API Planifium (vérifiées le 2026-11-10). L'API ne publie aucun horaire pour ces cours : les horaires, les places et les trois cours TST-… sont des données de test préparées au format du guide. La date « données du … » est la date d'import de la copie.*

1. Critères d'acceptation par exigence

Exigence	Critère d'acceptation vérifiable	Tests
E1 Offre commune	Le catalogue affiche cours **et** offres des organisations dans une même liste ; chaque activité montre type, crédits, charge, disponibilité et provenance ; une valeur absente s'affiche « inconnue ».	T01, T02
E2 Suggestions expliquées	Chaque suggestion affiche au moins un critère qui l'explique ; modifier un intérêt change l'ordre ; **chaque curseur de préférence a un effet visible ou indique qu'il n'en a pas**.	T03, T22
E3 Préalables	Le résultat est l'un de trois : satisfait, non satisfait, impossible à vérifier. Un relevé absent ou une condition `manual` ne donne **jamais** « satisfait » ni « non satisfait ».	T04, T05, T06
E4 Contraintes avant modification	Avant l'envoi, le plafond est calculé **avec les demandes en attente** ; un dépassement est affiché et la demande n'est pas envoyée sans décision de l'étudiant.	T07, T08
E5 Catégories et blocs	Chaque activité indique la catégorie à laquelle elle contribue ; les exigences restantes sont calculées par catégorie.	T09
E6 États des demandes	Une demande suit uniquement les transitions du modèle d'états ; la date et l'interlocuteur du dernier changement sont affichés ; les crédits ne sont « en cours » qu'à l'état CONFIRMEE.	T10, T11, T12
E7 Dates utiles	Une date inconnue s'affiche « date non publiée » ; un rappel est créé 7 jours avant une échéance connue, une seule fois.	T13
E8 Retrait expliqué	Les conséquences connues (crédits, frais, temps plein) sont affichées **avant** la confirmation ; annuler ne modifie rien.	T14
E9 Seuil de temps plein	Passer sous 12 crédits affiche un avertissement et une ressource à contacter, sans annoncer une perte d'aide financière.	T15
E10 Types de crédits	Acquis, en cours et planifiés sont affichés séparément ; une intention ne change que les planifiés.	T16

E11 Intentions à réexaminer	Un changement détecté par la synchronisation marque l'intention « à réexaminer », avec la session et le motif ; elle n'est pas supprimée ; l'absence de données n'est pas un changement.	T17
E12 Effets des appréciations	Une appréciation ne modifie jamais la disponibilité déclarée sans l'accord de l'étudiant ; **son effet sur les suggestions est expliqué au moment où il se produit**.	T18, T23
E13 Alternatives et conseil	Une activité inaccessible affiche au moins une alternative connue ou un interlocuteur.	T05, T19
E14 Bilan final	Le bilan classe chaque élément en satisfait, incomplet ou incertain.	T20
R5 Authentification et autorisation *(révision)*	Sans jeton valide, l'API refuse (401) ; un utilisateur ne peut agir que sur ses propres données, selon la matrice des droits (403).	T21, T24
R6 Synchronisation *(révision)*	Les places et les cours affichés portent la date de la copie Planifium.	T02, T19

2. Jeux de données

Ils reprennent les profils utilisés pendant l'enquête de la phase 1.

Jeu	Contenu
JD-30 « Profil 30 h »	Étudiant `E30`, disponibilité 30 h/sem, programme avec plafond de 18 crédits et seuil de temps plein à 12 ; relevé importé le 2026-09-02 : IFT1015 réussi. Curseurs au milieu, intérêt principal « Logiciel ».
JD-8 « Profil 8 h »	Étudiant `E8`, disponibilité 8 h/sem, plafond de 18 crédits ; **aucun relevé importé**.
JD-40 « Profil 40 h »	Étudiant `E40`, 40 h/sem ; plan avec une case « Projet supervisé ou stage » libre (0 sur 1) et une case « Séminaires, concours ou stages » libre (0 sur 1).
PL-COURS (copies Planifium, format du guide)	**IFT1025** *(réponse réelle de l'API)* : « Programmation 2 », 3 cr, `requirement_text` « IFT1015 ou IFT1016 », `requirements.prerequisites` = `any( course IFT1015, course IFT1016 )`, `status: "exact"`, `equivalent_courses` et `unparsed` vides, `available_terms` automne, hiver et été, **`schedules` vide** ; copie importée le 2026-10-20. **IFT3395** *(réponse réelle)* : « Fondements de l'apprentissage machine », 3 cr, `status: "exact"`, exigence `all( IFT2125, MAT1600, any( MAT1978, all( MAT1720, STT1700 ) ) )`, équivalent IFT3390, offert **à l'automne seulement**, `schedules` vide. **IFT2255** *(réponse réelle)* : « Génie logiciel », 3 cr, `status: "exact"`, exigence IFT1025, offert automne et été, `schedules` vide. **STT1700** *(réponse réelle)* : « Introduction à la statistique », 3 cr, `status: "none"` (aucune exigence), `schedules` vide. **Cours de test** *(inventés, non Planifium)* : **TST-MAN** avec `status: "manual"` (« permission du professeur »), **TST-EQV** avec l'exigence « IFT1015 ou compétence équivalente » (`any` avec une condition `manual`), **TST-GPA** avec une moyenne

	minimale (gpa 3,0). **Horaires de test** : une section d'IFT2255 avec `capacity` 120, `enrolled` 120, `updated_at` 2026-10-20 ; l'API réelle ne publie aucun horaire pour ces cours.
ORG (offres)	Projet « Données et IA » (3 cr, 8 h, encadrant `ENC-A`) ; Stage « Fondation Arpent » (6 cr, 16 h, organisation `ORG-1`) ; Stage « Coopérative Volta » (6 cr, échéance de réponse 2026-11-10).
CAL (calendrier importé)	Abandon avec frais à partir du 2026-09-16 ; retrait refusé à partir du 2026-11-06 *(règle visée, à valider, voir R8)* ; échéance d'abandon le 2026-11-15 ; concours « Sprint de design » : **aucune date**.

3. Cas de test

`ReglePrealables` évalue l'arbre `requirements` de la **copie datée** de Planifium, sans appeler l'API à chaque demande. La mention « même résultat que Planifium : `yes` / `no` / `unknown` » indique, à titre de repère, ce que donnerait l'évaluation de Planifium sur les mêmes données.

T01 — Catalogue commun (N) · E1

- **Objectif :** vérifier que cours et offres apparaissent ensemble avec leurs informations.
- **Préconditions :** PL-COURS et ORG chargés ; JD-30 connecté.
- **Données :** IFT1025, STT1700, projet « Données et IA » ; la charge de STT1700 est absente.
- **Étapes :** 1. Ouvrir le catalogue. 2. Filtrer sur « toutes les activités ».
- **Résultat attendu :** les 3 activités sont dans la même liste, avec type, crédits et provenance (« Planifium, copie importée le 2026-10-20 » ou « organisation ») ; la charge de STT1700 s'affiche « inconnue » ; IFT1025 indique « aucun horaire publié » (son `schedules` est vide).

T02 — Planifium indisponible (É) · E1, R6

- **Objectif :** vérifier que le catalogue reste utilisable avec la dernière copie quand la source ne répond pas.
- **Préconditions :** une copie de IFT1025 datée du 2026-10-20 existe ; Planifium ne répond pas (délai dépassé).
- **Données :** PL-COURS.
- **Étapes :** 1. Lancer la synchronisation. 2. Ouvrir la fiche de IFT1025.
- **Résultat attendu :** aucune erreur pour l'étudiant ; la fiche affiche « copie importée le 2026-10-20, à vérifier » ; la copie n'est ni effacée ni remplacée.

T03 — Suggestions expliquées (N) · E2

- **Objectif :** vérifier qu'une suggestion est expliquée et suit les intérêts.
- **Préconditions :** JD-30, intérêt « données » sélectionné.
- **Données :** ORG, PL-COURS.
- **Étapes :** 1. Ouvrir les suggestions. 2. Remplacer l'intérêt « données » par « robotique ». 3. Rouvrir les suggestions.
- **Résultat attendu :** à l'étape 1, chaque suggestion affiche au moins un critère (ex. « correspond à votre intérêt : données ») ; à l'étape 3, l'ordre des suggestions a changé.

T04 — Préalable satisfait (N) · E3

- **Objectif :** vérifier qu'un préalable réussi permet la demande.
- **Préconditions :** JD-30 (IFT1015 réussi) ; case Cours libre.
- **Données :** IFT1025 (`any` IFT1015 / IFT1016, `exact`).
- **Étapes :** 1. Demander IFT1025.
- **Résultat attendu :** `ReglePrealables` → SATISFAIT (même résultat que Planifium : `yes`) ; la demande est créée à l'état EN_ATTENTE ; les 3 crédits s'affichent « en demande ».

T05 — Préalable manquant (É) · E3, E13

- **Objectif :** vérifier qu'un préalable non réussi bloque la demande (cas IFT1025 observé en phase 1).
- **Préconditions :** relevé de JD-30 modifié pour ne contenir que MAT1978.
- **Données :** cas A : IFT1025 ; cas B : IFT3395 (arbre imbriqué : `all( IFT2125, MAT1600, any( MAT1978, all( MAT1720, STT1700 ) ) )`), avec un relevé contenant IFT2125, MAT1600 et MAT1720.
- **Étapes :** 1. Cas A : demander IFT1025. 2. Cas B : demander IFT3395 ; puis ajouter STT1700 au relevé et recommencer.
- **Résultat attendu :** cas A : NON_SATISFAIT (même résultat que Planifium : `no`), message « préalable manquant : IFT1015 ou IFT1016 » tiré de `missing_courses` ; **aucune demande n'est créée** ; au moins une alternative est proposée. Cas B : NON_SATISFAIT, le message désigne la branche non satisfaite (MAT1978, ou STT1700 avec MAT1720) ; après l'ajout de STT1700, SATISFAIT.

T06 — Préalable impossible à vérifier (L) · E3

- **Objectif :** vérifier qu'une donnée manquante ou non interprétable ne devient jamais une validation, ni un refus.
- **Préconditions :** JD-8 (aucun relevé) pour le cas A ; JD-30 pour les cas B, C et D.
- **Données :** cas A : IFT1025 ; cas B : cours de test TST-MAN (condition `manual`) ; cas C : cours de test TST-EQV (« IFT1015 ou compétence équivalente ») ; cas D : cours de test TST-GPA (moyenne minimale 3,0), sans moyenne fournie.
- **Étapes :**
  1. Cas A : JD-8 demande IFT1025.
  2. Cas B : JD-30 demande TST-MAN.
  3. Cas C : retirer IFT1015 du relevé de JD-30 et demander le cours du cas C ; puis remettre IFT1015 et recommencer.
  4. Cas D : JD-30 (aucune moyenne enregistrée) demande le cours du cas D.
- **Résultat attendu :**
  - Cas A : la plateforme (`Etudiant.estReleveComplet()` est faux) **ne traite pas l'absence de relevé comme une liste vide** (une liste vide donnerait « non » dans Planifium) ; résultat INCONNU, « relevé non importé : préalable impossible à vérifier » ; la demande peut être envoyée avec cet avertissement.

- Cas B : INCONNU (même résultat que Planifium : `unknown`) ; le texte « permission du professeur » est affiché dans les points à vérifier (`to_verify`).
- Cas C : sans IFT1015, INCONNU (et **non** NON_SATISFAIT) ; avec IFT1015, SATISFAIT.
- Cas D : INCONNU, « moyenne non fournie : condition à vérifier » ; la plateforme n'invente pas de moyenne.
- Dans aucun cas le résultat n'est affiché « satisfait » quand une information manque.

T07 — Exactement au plafond (L) · E4

- **Objectif :** vérifier qu'atteindre le plafond n'est pas un dépassement.
- **Préconditions :** JD-30 ; 15 crédits confirmés ; aucune demande en attente.
- **Données :** STT1700 (3 cr).
- **Étapes :** 1. Demander STT1700.
- **Résultat attendu :** 15 + 3 = 18 = plafond → `ReglePlafondCredits` SATISFAIT ; STT1700 n'a aucune exigence (statut `none`), donc `ReglePrealables` ne bloque pas ; demande EN_ATTENTE ; aucun avertissement de plafond.

T08 — Dépassement avec demandes en attente (É) · E4

- **Objectif :** vérifier que les demandes en attente comptent dans le plafond (correction de l'observation 25/18).
- **Préconditions :** JD-30 ; 15 crédits confirmés + IFT1025 (3 cr) EN_ATTENTE.
- **Données :** STT1700 (3 cr).
- **Étapes :** 1. Demander STT1700. 2. Choisir « garder en brouillon ».
- **Résultat attendu :** l'étape 1 affiche « 21 crédits sur un plafond de 18 (dont 3 en attente) » et **aucune demande n'est envoyée** ; à l'étape 2, un brouillon est enregistré **dans le plan** (ce n'est pas un état de `Demande`), la contrainte reste visible et le statut « brouillon » est affiché.

T09 — Catégorie d'un stage (N) · E5

- **Objectif :** vérifier qu'un stage compte dans la catégorie annoncée (cas Fondation Arpent, phase 1).
- **Préconditions :** JD-40 ; case « Projet supervisé ou stage » libre (0 sur 1) ; case « Séminaires, concours ou stages » libre (0 sur 1).
- **Données :** Stage « Fondation Arpent ».
- **Étapes :** 1. Ouvrir la fiche du stage. 2. Demander le stage. 3. Faire confirmer la demande.
- **Résultat attendu :** le bouton « Demander » est actif ; la fiche indique « contribue à : Projet supervisé ou stage » ; après la confirmation, cette catégorie passe de « 0 sur 1 » à « 1 sur 1 » et la catégorie « Séminaires, concours ou stages » reste à « 0 sur 1 ».

T10 — Cycle complet d'une demande (N) · E6

- **Objectif :** vérifier les états et le moment où les crédits sont comptés.
- **Préconditions :** JD-30 ; projet « Données et IA » demandé (EN_ATTENTE).
- **Données :** ORG, encadrant `ENC-A`.
- **Étapes :** 1. `ENC-A` accepte. 2. L'étudiant ouvre ses demandes. 3. L'étudiant accepte l'offre.

- **Résultat attendu :** après l'étape 1, l'état est OFFRE_RECUE et les crédits restent « en demande » ; l'étape 2 affiche la date du changement et « interlocuteur : ENC-A » ; après l'étape 3, l'état est CONFIRMEE et les 3 crédits passent « en cours » ; 3 `ChangementEtat` sont enregistrés au total (création, offre reçue, confirmée).

T11 — Expiration, sans doublon de notification (É) · E6, R7

- **Objectif :** vérifier l'expiration automatique et l'absence de doublon.
- **Préconditions :** demande au stage « Coopérative Volta » EN_ATTENTE, échéance de réponse 2026-11-10.
- **Données :** horloge de test fixée au 2026-11-11 à 01 h.
- **Étapes :** 1. Lancer le planificateur. 2. Le relancer une seconde fois.
- **Résultat attendu :** l'état est EXPIREE ; l'étudiant a **exactement une** notification « demande expirée » (même clé unique) ; la seconde exécution ne change rien.

T12 — Transition interdite (É) · E6

- **Objectif :** vérifier qu'une demande ne peut pas sauter l'offre.
- **Préconditions :** une demande EN_ATTENTE.
- **Données :** tentative de transition vers CONFIRMEE par un appel direct à `Demande.transitionner`.
- **Étapes :** 1. Appeler la transition directement (test unitaire de `Demande`).
- **Résultat attendu :** la transition est refusée (erreur « transition interdite ») ; l'état reste EN_ATTENTE ; aucun `ChangementEtat` n'est créé.

T13 — Échéances : inconnue, J-8, J-7, répétition (L) · E7

- **Objectif :** vérifier l'affichage des dates et la limite du rappel.
- **Préconditions :** CAL importé ; échéance d'abandon le 2026-11-15.
- **Données :** horloge à J-8 (2026-11-07), puis à J-7 (2026-11-08).
- **Étapes :** 1. Ouvrir la fiche du concours « Sprint de design ». 2. Lancer le planificateur à J-8. 3. Le lancer à J-7. 4. Le relancer une seconde fois à J-7.
- **Résultat attendu :** étape 1 : « date non publiée » et la personne à contacter ; étape 2 : aucun rappel ; étape 3 : un rappel « échéance dans 7 jours » avec ses conséquences ; étape 4 : toujours **un seul** rappel.

T14 — Retrait expliqué, annulé, puis refusé après le 6 novembre (N/L) · E8

- **Objectif :** vérifier que les conséquences sont présentées avant toute modification, et le comportement visé après la date limite.
- **Préconditions :** JD-30 ; 15 crédits confirmés ; STT1700 CONFIRMEE.
- **Données :** retrait de STT1700 (3 cr) ; cas A : horloge au 2026-10-15 (après l'abandon avec frais, avant le 6 novembre) ; cas B : horloge au 2026-11-09.
- **Étapes :** cas A : 1. Demander le retrait. 2. Lire l'écran de confirmation. 3. Cliquer « Annuler ». Cas B : 4. Demander le retrait.
- **Résultat attendu :** cas A, étape 2 : « 12 crédits après le retrait », « abandon avec frais (source : calendrier) » et les démarches au Registrariat ; après l'étape 3, le plan est identique (15 crédits,

STT1700 CONFIRMEE). Cas B : le retrait est **refusé** et l'étudiant est orienté vers le Registrariat *(règle visée, à valider ; le prototype retirait sans confirmation)*.

T15 — Seuil de temps plein, limite exacte (L) · E9

- **Objectif :** vérifier l'avertissement au bon seuil.
- **Préconditions :** JD-30, seuil de 12 crédits.
- **Données :** cas A : 15 → 12 crédits ; cas B : 12 → 9 crédits.
- **Étapes :** 1. Simuler le retrait du cas A. 2. Simuler le retrait du cas B.
- **Résultat attendu :** cas A : aucun avertissement de temps plein (12 n'est pas sous 12) ; cas B : « votre plan passerait sous le seuil de temps plein (9 sur 12) », avec une ressource à contacter ; **aucun** message n'annonce une perte d'aide financière.

T16 — Intention future et crédits (N) · E10

- **Objectif :** vérifier que l'intention ne change que les crédits planifiés.
- **Préconditions :** JD-30 : acquis 3, en cours 15, planifiés 0.
- **Données :** intention IFT3395 pour l'automne 2027 (IFT3395 n'est offert qu'à l'automne).
- **Étapes :** 1. Planifier l'intention. 2. Consulter les compteurs.
- **Résultat attendu :** acquis 3, en cours 15, planifiés 3 ; le total « 21 » est affiché comme « projection ».

T17 — Réexamen après synchronisation (N/L) · E11

- **Objectif :** vérifier qu'un changement marque l'intention sans la supprimer, seulement pour la bonne session, et que l'absence de données n'est pas un changement.
- **Préconditions :** intention IFT3395 pour l'automne 2027 ; la copie précédente (horaires de test) listait une section de IFT3395 à l'automne 2027.
- **Données :** cas A : la nouvelle copie ne liste plus de section de IFT3395 à l'automne 2027 ; cas B : IFT3395 est seulement complet à l'automne 2026 ; cas C : la nouvelle copie ne contient **aucun horaire** pour l'automne 2027 (offre pas encore publiée ; c'est l'état réel actuel de l'API, qui ne publie aucun horaire pour ces cours).
- **Étapes :** 1. Synchroniser avec le cas A. 2. Remettre l'intention, puis synchroniser avec le cas B. 3. Remettre l'intention, puis synchroniser avec le cas C.
- **Résultat attendu :** cas A : l'intention est conservée, marquée « à réexaminer — automne 2027 — n'est plus offert » et une notification est créée ; cas B : l'intention **n'est pas** marquée (une indisponibilité actuelle ne s'étend pas aux sessions futures) ; cas C : l'intention reste « planifiée », non marquée, avec la mention « offre non publiée » (aucune donnée n'est traitée comme un retrait).

T18 — Appréciation sans effet caché (É) · E12

- **Objectif :** vérifier qu'une appréciation ne modifie pas la disponibilité en silence.
- **Préconditions :** JD-30, disponibilité 30 h.
- **Données :** appréciation « Peu pertinent » sur le projet « Données et IA ».
- **Étapes :** 1. Enregistrer l'appréciation. 2. Ouvrir le profil. 3. Refuser le changement proposé.

- **Résultat attendu :** une explication de l'effet sur les suggestions s'affiche ; la disponibilité est toujours de 30 h ; un éventuel changement proposé n'est appliqué qu'après acceptation (refusé ici, donc rien ne change).

T19 — Alternatives à une activité complète (N) · E13

- **Objectif :** vérifier qu'une activité inaccessible propose une suite.
- **Préconditions :** JD-30.
- **Données :** IFT2255 (section avec `capacity` 120 et `enrolled` 120), autre section connue sur un autre campus.
- **Étapes :** 1. Ouvrir IFT2255. 2. Simuler une synchronisation où la section garde `enrolled` = `capacity` avec `updated_at` 2026-11-03, puis rouvrir IFT2255.
- **Résultat attendu :** étape 1 : « complet (copie du 2026-10-20) » ; au moins une alternative (autre section, autre campus ou autre session) et un interlocuteur sont affichés ; aucune équivalence non validée n'est présentée comme acquise. Étape 2 : « complet selon les données du 2026-11-03 » ; la date affichée a changé et l'historique garde l'ancienne copie.

T20 — Bilan de fin de période (N) · E14

- **Objectif :** vérifier le classement du bilan.
- **Préconditions :** JD-30 en fin de session : 2 activités CONFIRMEE, 1 demande EN_ATTENTE, 1 intention « à réexaminer », 1 préalable INCONNU.
- **Données :** fin de session simulée.
- **Étapes :** 1. Demander le bilan.
- **Résultat attendu :** satisfaits = 2 activités ; incomplets = la demande en attente et les exigences restantes ; incertains = l'intention à réexaminer et le préalable inconnu.

T21 — Autorisation refusée (É) · R5

- **Objectif :** vérifier la matrice des droits.
- **Préconditions :** demande au projet de `ENC-A` ; un autre encadrant `ENC-B` est connecté.
- **Données :** jetons valides de `ENC-B` et de `E30`.
- **Étapes :** 1. `ENC-B` tente de répondre à la demande. 2. `E30` tente de modifier le plafond du programme.
- **Résultat attendu :** les deux actions sont refusées (HTTP 403) ; la demande et la règle sont inchangées ; les deux tentatives sont journalisées.

T22 — Chaque curseur a un effet, ou le dit (É) · E2

- **Objectif :** vérifier qu'aucun curseur de préférence n'est ignoré en silence (en phase 1, « Expérience de travail » n'avait aucun effet observable).
- **Préconditions :** JD-30 avec les trois curseurs (Concentration, Approche, Expérience de travail) au milieu ; au moins 6 activités dans ORG et PL-COURS, dont des stages et des projets.
- **Données :** suggestions de départ notées (ordre complet).

- **Étapes :** 1. Noter l'ordre des suggestions. 2. Mettre « Concentration » à « Se concentrer » et noter l'ordre. 3. Remettre au milieu, mettre « Approche » à « Pratique et appliqué » et noter l'ordre. 4. Remettre au milieu, mettre « Expérience de travail » à « Prioritaire » et noter l'ordre.
- **Résultat attendu :** pour **chacun** des trois curseurs, soit l'ordre change et un critère l'explique, soit un message indique que ce curseur n'a pas d'effet sur les suggestions actuelles et pourquoi. Un curseur sans changement **et** sans message est un échec du test. Avec « Expérience de travail » à « Prioritaire », les stages et projets passent avant les cours.

T23 — Portée d'une appréciation « Peu pertinent » (N) · E12

- **Objectif :** vérifier qu'une appréciation sur une seule activité explique tout effet sur les autres (constat S11).
- **Préconditions :** JD-30 ; intérêt principal « Logiciel » ; IFT1025 suivi ; au moins 3 autres offres liées à « Logiciel » dans les suggestions.
- **Données :** appréciation « Peu pertinent » sur IFT1025, puis « Charge trop lourde » (seule) sur une autre activité suivie.
- **Étapes :** 1. Noter l'ordre. 2. Donner « Peu pertinent » à IFT1025. 3. Lire le message et noter l'ordre. 4. Donner « Charge trop lourde » seule à l'autre activité. 5. Lire le message et noter l'ordre.
- **Résultat attendu :** à l'étape 3, si les autres offres de « Logiciel » reculent, le message le dit explicitement (« les offres liées à Logiciel sont moins suggérées ») ; à l'étape 5, soit l'ordre change avec une explication, soit le message indique que cette appréciation n'a pas d'effet sur le classement. Aucun changement d'ordre sans explication.

T24 — Authentification : jeton absent ou expiré (É) · R5

- **Objectif :** vérifier que l'API refuse toute requête non authentifiée.
- **Préconditions :** API démarrée ; plan de `E30` existant.
- **Données :** requête de lecture du plan de `E30` : sans jeton ; avec un jeton expiré ; avec un jeton valide.
- **Étapes :** 1. Envoyer la requête sans jeton. 2. L'envoyer avec le jeton expiré. 3. L'envoyer avec le jeton valide de `E30`.
- **Résultat attendu :** étapes 1 et 2 : HTTP 401, aucune donnée du plan dans la réponse, tentatives journalisées, l'application web renvoie l'utilisateur vers la connexion ; étape 3 : HTTP 200 avec le plan de `E30`.

4. Couverture

Type	Tests
Nominal	T01, T03, T04, T09, T10, T14, T16, T17, T19, T20, T23
Limite	T06, T07, T13, T14, T15, T17
Échec	T02, T05, T08, T11, T12, T18, T21, T22, T24

T14 et T17 combinent deux types (ils sont comptés dans les deux lignes). Toutes les exigences E1 à E14 ont au moins un test. Les révisions R5 (authentification et autorisation), R6 (synchronisation) et R7 (notifications) sont aussi testées.

6. B4 — Modèle de données

Ce modèle classe les données de la plateforme selon leur rôle, indique où chaque donnée conservée est stockée, et rattache chaque regroupement à un composant de B1. Les noms de classes sont ceux de B2.

1. Les trois familles de données

Famille	Définition (pour notre projet)	Durée de vie
Échange	Données qui circulent entre deux parties : application ↔ API, API ↔ Planifium, fichiers importés, courriels.	Le temps d'une requête ou d'un import.
Préservation	Données conservées d'une visite à l'autre : état, historique, copies datées des sources.	Persistantes.
Présentation	Données calculées ou mises en forme pour l'écran : badges, compteurs, messages.	Le temps de l'affichage ; recalculées à chaque fois.

2. Données d'échange

Donnée	Entre	Format	Composant B1
Requête de demande `{activiteId}` → réponse `{demandeId, etat, resultatsRegles[]}`	Application web ↔ API	JSON / HTTPS	Routeur REST, Service Demandes
Aperçu de retrait `{creditsApres, frais, sousSeuilTempsPlein, demarches[]}`	API → Application web	JSON / HTTPS	Service Échéances et retraits
Réponse d'un encadrant `{demandeId, accepte, conditions, echeance}`	Application web → API	JSON / HTTPS	Service Demandes
Jeton d'identité	Fournisseur d'identité → Application web → API	OpenID Connect (JWT)	Contrôle d'accès
Réponse Planifium d'un cours : `requirements`, `status`, `updated_at`, `capacity`, `enrolled`	API Planifium → Client Planifium	JSON (REST)	Client Planifium
Relevé de notes, calendrier	Fichier → Importeur	Fichier importé	Importeur de fichiers
Alerte urgente	API → Service de courriel	SMTP	Service Notifications
Export d'horaire	API → étudiant	iCalendar (ICS)	Service Plan

3. Données de préservation (conservées d'une visite à l'autre)

Classe conservée (B2)	Où	Ce qui est gardé	Composant B1
Etudiant, Appreciation	**Serveur** (PostgreSQL)	Profil, disponibilité, préférences, appréciations	Service Profil et suggestions
Relevé importé	**Serveur** (PostgreSQL) + **fichier** d'origine sur le serveur	Cours réussis, date d'import, fichier source pour la provenance	Importeur de fichiers
PlanSession, Case, Intention	**Serveur**	Plan, brouillon, intentions, marque « à réexaminer »	Service Plan
Demande, ChangementEtat	**Serveur**	État courant et **historique**	Service Demandes

		complet des transitions (date, auteur)	
OffreActivite	**Serveur**	Offres publiées par les organisations (la plateforme est leur source de vérité)	Service Catalogue
Programme, RegleProgramme	**Serveur**	Règles **versionnées** (qui les a changées et quand)	Moteur de règles du programme
CopiePlanifium (copies des réponses de l'API)	**Serveur**	JSON brut, `updated_at` de Planifium, date d'import, statut d'analyse, marque « à vérifier » ; la copie précédente est gardée pour la comparaison	Client Planifium (produit et enregistre) ; utilisée par Service Synchronisation (compare) et Service Catalogue (affiche la date)
Echeance	**Serveur**	Dates importées du calendrier, avec leur source	Service Échéances et retraits
Notification	**Serveur**	Destinataire, événement, clé unique (anti-doublon), lue / non lue	Service Notifications
Utilisateur, journal	**Serveur**	Rôle de chaque compte ; journal des écritures et des refus	Contrôle d'accès
Jeton de session	**Navigateur** (cookie `HttpOnly`)	Identité de la session seulement, jamais de données métier	Contrôle d'accès
Préférences d'affichage (filtre, onglet)	**Navigateur** (`localStorage`)	Confort d'affichage uniquement ; leur perte est sans conséquence	Application web

Aucune donnée métier (plan, demandes, règles) n'est conservée dans le navigateur. C'est un changement par rapport au prototype de la phase 1, qui gardait tout dans le `localStorage` : vider le navigateur effaçait le plan, et deux appareils montraient deux plans différents.

Les copies Planifium ne sont jamais envoyées telles quelles au navigateur. L'API les traduit d'abord en données de présentation, avec leur date.

Toutes les données de préservation passent par les Dépôts de données, sauf les copies et les fichiers importés, que les connecteurs écrivent directement (relations « Copies datées » et « Relevés, dates, provenance » de B1).

4. Données de présentation (calculées, jamais stockées)

Donnée affichée	Calculée à partir de	Composant B1
Badge d'état d'une demande (« en attente », « offre à accepter avant le… »)	Demande, ChangementEtat	Service Demandes
Compteurs acquis / en cours / en demande / planifiés, et « projection »	PlanSession, Intention, relevé	Service Plan
Résultat d'une règle (✓ satisfait, ✗ non satisfait, ? impossible à vérifier)	ResultatRegle	Moteur de règles du programme
« Données du 2026-10-20, à vérifier »	CopiePlanifium (`updated_at`, statut)	Service Catalogue
« date non publiée »	Echeance sans date	Service Échéances et retraits
Explication d'une suggestion	Profil + critères de classement	Service Profil et suggestions
Bilan satisfait / incomplet / incertain	Plan, demandes, intentions, règles	Service Bilan

5. Parcours de deux données importantes, de la source à l'affichage

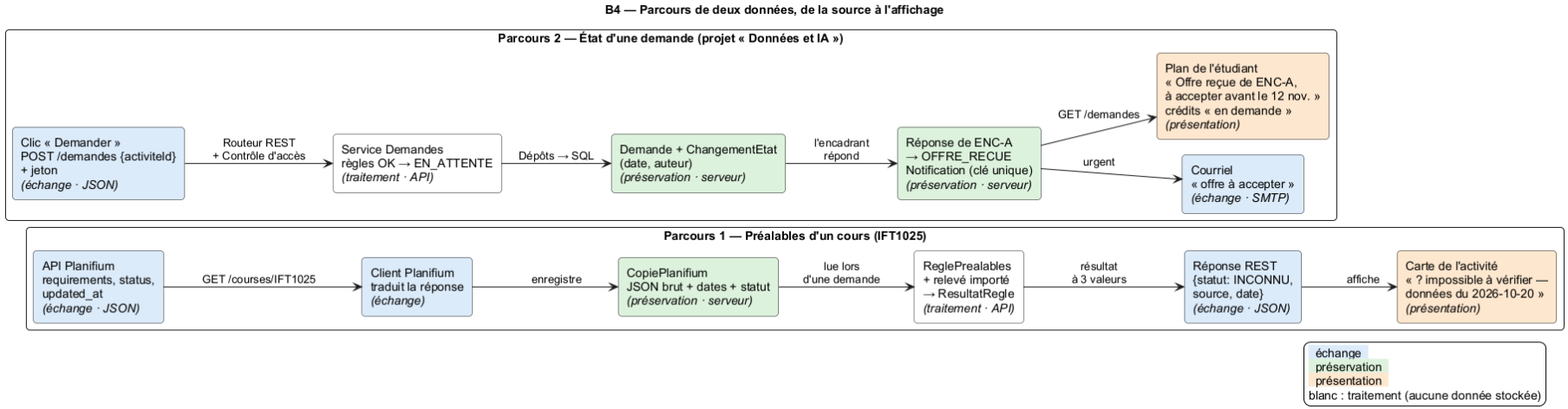


Figure 9 — (en bleu l'échange, en vert la préservation, en orange la présentation)

Parcours 1 — Les préalables d'un cours (IFT1025)

9. **Source (échange)** : Planifium renvoie pour IFT1025 `requirements` (`any( IFT1015, IFT1016 )`, texte « IFT1015 ou IFT1016 »), `status: "exact"` et un `schedules` vide *(Client Planifium)*.
10. **Préservation** : le Client Planifium enregistre une `CopiePlanifium` (JSON brut, date d'import, statut). Si `status` vaut `manual`, la copie est marquée « à vérifier » *(Client Planifium → base de données)*.
11. **Traitement** : quand l'étudiant demande IFT1025, `ReglePrealables` compare les exigences au relevé importé → `ResultatRegle` = SATISFAIT, NON_SATISFAIT ou INCONNU *(Moteur de règles du programme)*.
12. **Échange** : l'API renvoie `{regle: "Préalables", statut: "INCONNU", message, source: "Planifium, 2026-10-20"}` *(Routeur REST)*.
13. **Présentation** : la carte affiche « ? Préalable impossible à vérifier — données du 2026-10-20 » *(Application web)*.

Une exigence non comprise par Planifium, ou un relevé absent, arrive à l'écran comme « ? », jamais comme « ✓ ». C'est la correction de l'observation IFT1025 de la phase 1.

Parcours 2 — L'état d'une demande (projet « Données et IA »)

14. **Source (échange)** : l'étudiant clique sur « Demander » → `POST /demandes {activiteId}` avec son jeton *(Application web → Routeur REST → Contrôle d'accès)*.
15. **Traitement** : le Service Demandes fait vérifier les contraintes par le Moteur de règles, puis crée la `Demande` à l'état EN_ATTENTE *(Service Demandes)*.
16. **Préservation** : la base enregistre `Demande` et un `ChangementEtat` (→ EN_ATTENTE, date, auteur) *(Dépôts de données)*.
17. **Événement** : l'encadrant accepte → OFFRE_RECUE ; le Service Notifications enregistre une `Notification` (clé `demande-42-OFFRE_RECUE`) et envoie un courriel *(Service Notifications)*.
18. **Présentation** : le plan affiche « Offre reçue de ENC-A le 2026-11-05 — à accepter avant le 2026-11-12 » ; les crédits restent « en demande » jusqu'à CONFIRMEE *(Application web)*.

Parcours complémentaire — Les places disponibles et le réexamen d'une intention

`capacity` et `enrolled` d'une section (champ `schedules`) → `CopiePlanifium` → `Cours.placesDisponibles()` → « 0 place selon les données du 2026-11-03 ». La date est toujours affichée, puisque Planifium précise que les places peuvent avoir changé depuis `updated_at`. Quand `schedules` est vide (cas réel de IFT1025, IFT2255, IFT3395 et STT1700 au 2026-11-10), l'écran affiche « aucun horaire publié » : l'absence de données n'est ni « complet » ni « offert ».

Réexamen d'une intention future (E11). Lors de la synchronisation, `ServiceSynchronisation.comparer()` compare l'ancienne et la nouvelle `CopiePlanifium` d'un cours. Si l'offre visée par une `Intention` a changé (par exemple, le cours n'est plus offert à la session visée), le Service Plan appelle `Intention.marquerAReexaminer(motif)` : l'attribut `aReexaminer` passe à vrai et `motifReexamen` est renseigné, **sans supprimer l'intention** (préservation, serveur). Une notification est créée, et l'écran du plan affiche « à réexaminer — automne 2027 — n'est plus offert » (présentation). Une copie sans horaire publié n'est pas un changement : l'intention reste « planifiée » (voir B3, T17).

7. Défense des choix

Chaque choix est présenté avec l'option écartée, ce que nous gagnons et ce que nous acceptons de perdre.

#	Choix retenu	Option écartée	Ce que nous gagnons	Ce que nous acceptons de perdre
C1	**Les règles vivent dans l'API**, pas dans l'application web.	Garder les règles dans le navigateur, comme `rules.js` dans le prototype.	Les règles ne peuvent pas être contournées et s'appliquent de la même façon pour l'étudiant, l'encadrant et le planificateur.	Un serveur est nécessaire, et chaque vérification fait un aller-retour réseau.
C2	**Copies datées de Planifium**, synchronisées chaque nuit.	Appeler Planifium à chaque requête.	La plateforme fonctionne même si Planifium ne répond pas ; chaque donnée garde sa provenance et sa date.	Une donnée peut avoir jusqu'à un jour de retard. Nous le compensons en affichant toujours la date de la copie.
C3	**Résultat de règle à trois valeurs** : satisfait, non satisfait, inconnu.	Un résultat vrai / faux.	Une exigence non analysée par Planifium ou un relevé absent ne devient jamais une validation (E3, observation IFT1025).	L'interface est plus complexe, et l'étudiant voit parfois une incertitude au lieu d'une réponse nette.
C4	**L'offre est un état de la demande** (OFFRE_RECUE).	Deux classes, Demande et Offre, à synchroniser.	Une seule machine à états : une demande ne peut pas être à la fois « en attente » et « confirmée ». C'est la cohérence demandée par la correctrice.	La classe `Demande` porte plus d'attributs (échéance de l'offre, interlocuteur).
C5	**Une classe par règle du programme** (patron Stratégie).	Une table de configuration générique, ou un petit langage de règles.	Chaque règle se teste seule (B3 : T05 à T08, T15) et se retrouve facilement, comme l'exige l'énoncé.	Ajouter un nouveau type de règle demande du code, pas seulement une configuration.
C6	**Les demandes en attente comptent dans le plafond.**	Ne compter que les activités confirmées, comme le prototype.	On corrige le dépassement observé en phase 1 (25 crédits pour un plafond de 18).	Une demande en attente qui finit par être refusée bloque temporairement des crédits disponibles, mais l'étudiant peut annuler sa demande en attente à tout moment ou utiliser le mode brouillon pour tester d'autres combinaisons.
C7	**Notifications dans l'application et par courriel**, sans notification instantanée (*push*).	Notifications instantanées (WebSocket, notification mobile).	C'est simple, fiable, et la clé unique évite les doublons.	Une notification peut arriver avec quelques minutes de retard. Ce n'est pas grave pour des échéances qui se comptent en jours.
C8	**Un seul conteneur API découpé en composants**	Des microservices (un service par composant).	Déploiement simple sans latence réseau entre les services,	Impossible de faire passer à l'échelle un seul module

	(monolithe modulaire).		transactions SQL directes dans PostgreSQL et architecture réaliste à maintenir pour une équipe de quatre.	indépendamment (ex. Service Synchronisation), mais le découpage strict en composants (B1) permettra d'extraire un service plus tard sans toucher au modèle B2.
C9	**La plateforme ne modifie jamais une donnée institutionnelle.**	Permettre de corriger localement un relevé ou une date.	Pas de deuxième « vérité » qui contredirait le Registrariat ; les droits restent simples (matrice R2).	Une erreur dans une source reste visible jusqu'à sa correction à la source ; la plateforme peut seulement la signaler.

Limites assumées. Les dates d'abandon varient d'un cours à l'autre au Centre étudiant (R8) : elles sont des paramètres importés (`Echeance`), jamais codés en dur. Le prototype de la phase 1 n'implémente pas cette conception : c'est une conception cible, pas un système testé.

8. Déclaration de l'usage des outils d'IA

Conformément à l'énoncé, nous déclarons les outils employés et l'usage que nous en avons fait.

Outil	Usage	Artefacts touchés	Travail de l'équipe sur ce résultat
Claude (Anthropic), assistant de programmation	Première version des révisions de la phase 1 à partir de l'évaluation ; brouillons de B1 à B4 ; écriture des sources PlantUML ; vérification de la cohérence des noms entre artefacts ; assemblage du rapport	Révisions, B1, B2, B3, B4	B1 : Nada a repris le diagramme (lisibilité, technologies des relations, liens vers l'Importeur de fichiers) et écrit la matrice de traçabilité finale, avec les cas alternatifs. B2 : Ismail a redessiné les deux diagrammes (vue principale et annexe) dans Visual Paradigm à partir du brouillon PlantUML, avec l'accompagnement de Claude pour le dessin, puis a corrigé des erreurs de notation relevées par Claude. Il a rédigé lui-même les écarts et les choix de conception de B2 ; Claude a seulement vérifié leur cohérence avec le diagramme et la notation UML, et signalé les points à corriger. À la demande de Mohamed, Claude a ensuite appliqué dans Visual Paradigm les corrections de notation relevées en relecture (classes et opérations abstraites, types de retour, attribut `plafond`, coquilles). B3 : Hamza a vérifié les préalables réels dans Planifium (IFT1025, IFT3395, IFT2255, STT1700) et corrigé les tests en conséquence ; il a ajouté des cas d'échec sur les données manquantes et trois tests (curseurs, appréciation « Peu pertinent », authentification), retiré un doublon, et remplacé les horaires absents de l'API par des données de test annoncées comme telles (22 → 24 tests). B4, révisions et défense des choix : Mohamed a effectué la relecture critique et la validation de l'ensemble des révisions de la phase 1 (R1 à

			R8), du modèle de données (B4) et de la défense des choix architecturaux (C1 à C9). Plus précisément, il a vérifié et validé la matrice des droits des acteurs (R2) ainsi que la séparation stricte entre la persistance serveur (PostgreSQL) et le stockage client dans le navigateur (B4.3). Il a corrigé le statut des règles de dates limites d'abandon dans le tableau R8 (16 septembre et 6 novembre) pour refléter le fonctionnement réel du calendrier universitaire de l'UdeM (dates variables par cours paramétrées via l'entité `Echeance` plutôt que codées en dur), bonifié la section B4.5 en précisant l'impact de la synchronisation sur le parcours des intentions futures (`aReexaminer`), et reformulé les justifications et compromis des choix de conception C6 (gestion du plafond de crédits avec demandes en attente et mode brouillon) et C8 (architecture en monolithe modulaire versus microservices) afin d'en assurer la maîtrise complète lors de l'évaluation. Enfin, il a supervisé l'assemblage final du rapport et demandé les correctifs de notation UML dans Visual Paradigm pour B2. En équipe, nous avons harmonisé les noms des composants et des classes (Service Échéances et retraits, rattachement de `CopiePlanifium` aux connecteurs).
PlantUML	Rendu des diagrammes à partir de leur source texte (pour B2, brouillon seulement : les diagrammes remis sont ceux de Visual Paradigm)	A2.1, A2.2, C4 niveaux 1 à 3, B2 (brouillon), B4	—
ChatGPT (OpenAI)	B2 : comparaison de la conception avec les documents de la phase 1, vérification de classes et de	B2	B2 : Ismail a réécrit et vérifié le texte à partir de ses diagrammes et des documents de la phase 1.

	relations UML, reformulation des justifications écrites		
Visual Paradigm	Dessin des diagrammes de classes de B2 (vue principale et annexe)	B2	B2 : dessinés par Ismail.

Les choix de conception présentés dans la section « Défense des choix » ont été discutés et validés par l'équipe. Aucun résultat n'a été remis sans relecture.

9. Répartition du travail

Membre	Matricule	Responsabilités
Nada Chiki	20261599	B1 : C4 niveau 3 et matrice de traçabilité des cinq flux A2 ; question hebdomadaire à l'équipe enseignante
Ismail Mohib	20274803	B2 : diagramme de classes, modèle du domaine et écarts
Hamza Khalil	20259237	B3 : spécifications de tests, jeux de données et critères d'acceptation
Mohamed Driss	20205703	B4 : modèle de données ; révisions de la phase 1 ; défense des choix ; assemblage du rapport

Les noms communs (composants, classes, états d'une demande) ont été fixés en équipe, puis chaque partie a été relue par un autre membre (Nada ↔ Ismail pour B1 et B2, Hamza ↔ Mohamed pour B3 et B4).

10. Annexe

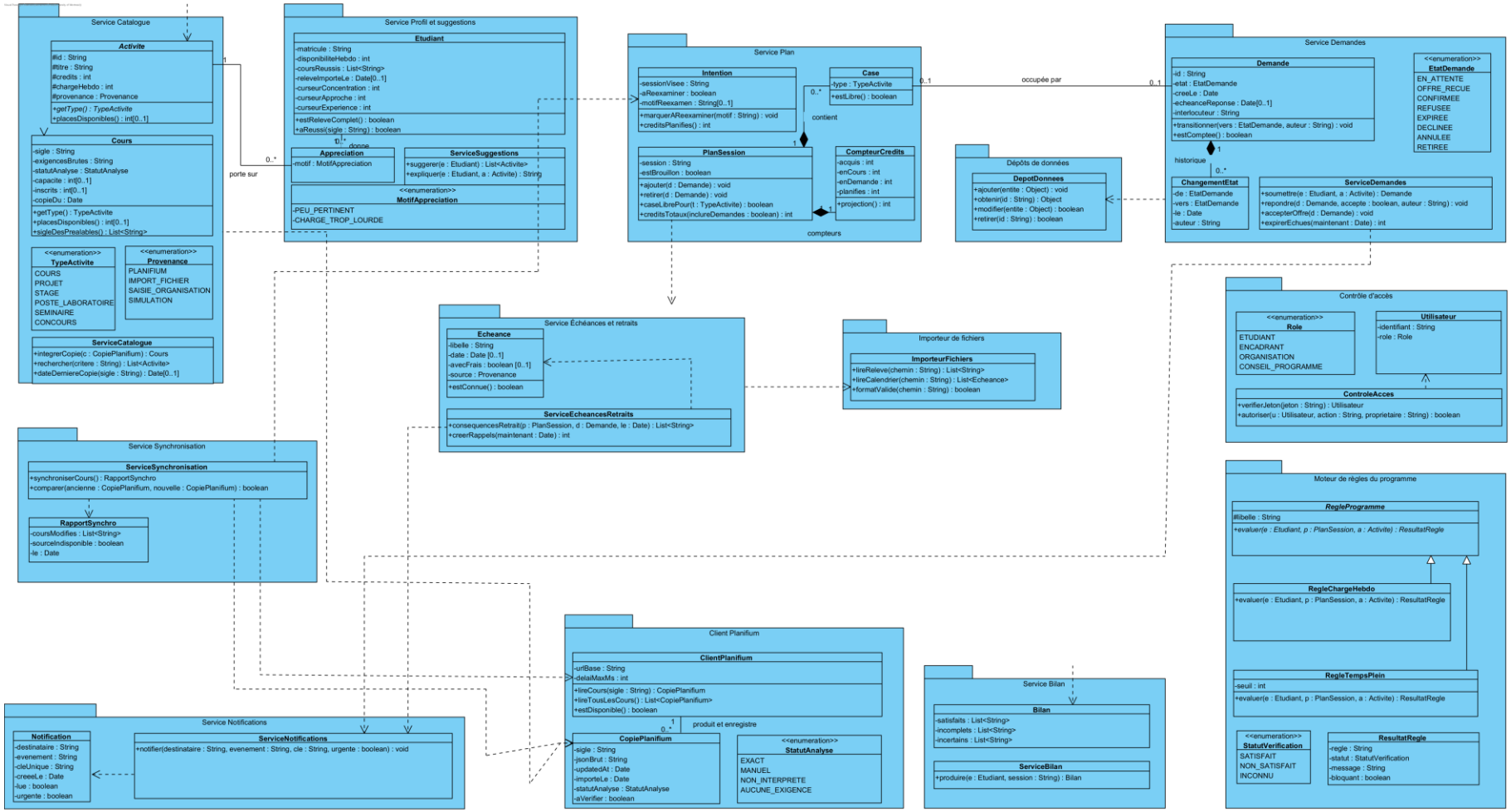


Figure 10 — B2 : classes complémentaires, rangées par composant de B1